
Active Causal Discovery as a Diagnostic Benchmark for LLM Agents

Sagar Deb
QpiAI

sagar.deb@qpiai.tech

Devam Shah
QpiAI

devam.s@qpiai.tech

Ashwanth Krishnan
QpiAI

ashwanth.krishnan@qpiai.tech

Abstract

Active causal discovery is a sequential task in which observations typically identify only a Markov-equivalence class, while interventions are needed to orient the remaining structure. We introduce the Active Causal Discovery Benchmark (ACDB), a controlled family of linear-Gaussian SCM instances with anonymized variables, a fixed observe-intervene-submit runtime, and a layered scoring contract that separates adjacency recovery, observationally identifiable orientation, and full directed-graph recovery. ACDB further calibrates evaluation with a random-floor-aware difficulty ladder and paired accepted-seed comparisons, so differences in directed performance are interpretable across methods and graph scales. Across five language models and five active policies, end-to-end `llm_raw` directed F1 averaged over L1-L5 ranges from 0.113 (Haiku-4.5) to 0.697 (GPT-5.5) against a classical `pc_greedy` reference of 0.689; supplying that reference as an observational prior raises the four weaker models by +0.28 to +0.37 directed F1, statistical-tool access alone does not, and the gap to the classical baseline widens with graph scale. Taken together, these results position ACDB as a benchmark for measuring observational, interventional, and end-to-end failures separately rather than compressing active causal discovery into a single graph score.

1 Introduction

Active causal discovery is a sequential inference problem: an agent must extract tentative structure from observational data, choose interventions under a finite budget, integrate the resulting evidence, and finally commit to a directed acyclic graph (DAG). Unlike purely observational causal discovery, the target is not fully determined by the observed distribution alone. In general, observational data identify a Markov equivalence class rather than a unique DAG, so additional interventions are required to resolve the remaining directional ambiguity [Spirtes et al., 2000, Verma and Pearl, 1990, Andersson et al., 1997, Eberhardt et al., 2005]. This makes active causal discovery a natural agent task rather than a static prediction problem.

Structural causal models (SCMs) make this domain especially well suited to diagnostic evaluation. They provide an exact hidden causal state together with precise intervention semantics, while CPDAG and equivalence-class theory make the observational ceiling explicit [Spirtes et al., 2000, Andersson et al., 1997, Hauser and Bühlmann, 2012]. The resulting separation is unusually clean: a system can fail by missing adjacencies, by failing to recover directions that are already compelled observationally, or by failing to use interventions to orient the remaining edges. In other words, SCM theory gives the evaluator access to distinct theoretical objects against which different parts of the task can be scored separately.

Most contemporary agent benchmarks instead report a single end-to-end success signal on complex environments such as software engineering, web navigation, and computer use [Jimenez et al., 2024, Liu et al., 2024, Zhou et al., 2024, Xie et al., 2024, Mialon et al., 2023]. Those evaluations are

valuable, but they are less diagnostic when the task itself contains separable inference stages. In causal discovery specifically, prior work has examined whether language models reason about causality at all [Kıcıman et al., 2023, Zečević et al., 2023], but a single graph score on the active task can still confound observational structure recovery, interventional reasoning, and final graph commitment. Benchmark conclusions can also become hard to interpret when graph families induce a non-trivial blind-random score floor. What is missing is not another leaderboard, but an evaluation instrument whose scoring layers and calibration are aligned with the underlying causal objects.

We introduce the **Active Causal Discovery Benchmark (ACDB)**, an SCM-grounded benchmark that fixes a hidden linear–Gaussian world generator, an observe–intervene–submit interaction protocol, and a paired seed map across agents. ACDB evaluates submissions with a layered scoring contract over skeleton recovery, observationally compelled orientation, full directed recovery, and structural error, and it calibrates its ladder with level-specific expected/random directed-F1 floors. We use this benchmark to compare classical baselines and LLM-based policies under the same active protocol, including end-to-end raw-data agents and hybrid policies that isolate observational structure recovery from interventional orientation.

On the current five-model, five-ablation panel, ACDB reveals several separable patterns. Layered scores expose distinctions that a single final-graph metric would hide; supplying a strong observational prior often yields large gains but does not remove the interventional orientation burden; and granting conditional-independence-test tools does not reliably improve directed recovery. More broadly, performance varies substantially across model families and degrades asymmetrically with graph complexity relative to the classical active baseline. These findings are not the benchmark’s primary contribution. Their role in this paper is to demonstrate that ACDB can localize failure to observational recovery, interventional orientation, or end-to-end integration.

Our contributions are:

- We introduce ACDB, a benchmark for active causal discovery with a fixed SCM generator, a fixed observe–intervene–submit protocol, and a theory-grounded layered scoring contract.
- We present a calibration methodology for active causal-discovery evaluation based on expected/random directed-F1 floors and a graph-scale ladder designed to avoid structurally guessable regions.
- We provide an empirical study across classical and LLM-based policies showing that the benchmark can localize failure across observational recovery, interventional orientation, and end-to-end graph integration.

2 Benchmark Setup

ACDB is organized around fixed benchmark instances rather than free-form causal-discovery episodes. Each instance defines a hidden causal world, a public data interface, and a benchmark-owned reference structure used later for scoring. This section makes that object concrete: what is sampled, what the agent sees, what remains hidden, and how accepted instances are assembled.

Task and instance family. Each instance begins from a hidden linear-Gaussian structural causal model (SCM) over observed variables $X_1, \dots, X_d$:

$$X_i = \sum_{j \in \text{Pa}(i)} w_{ij} X_j + \varepsilon_i, \quad \varepsilon_i \sim \mathcal{N}(0, \sigma_i^2), \quad (1)$$

where $\text{Pa}(i)$ are the parents of X_i in a directed acyclic graph (DAG) G and the noise terms are jointly independent. The SCM induces an observational distribution $P(X)$ together with interventional distributions under hard single-node actions $\text{do}(X_i = v)$, implemented by replacing the structural equation for X_i with the assignment $X_i := v$ [Spirtes et al., 2000, Eberhardt et al., 2005]. Benchmark instances are therefore not abstract graph puzzles: they are sampled causal worlds that produce both an observational data matrix and intervention-specific sample matrices under the same underlying SCM.

Observational data do not generally identify a unique DAG. Distinct DAGs can share the same skeleton and the same v-structures, and are therefore Markov equivalent [Verma and Pearl, 1990,

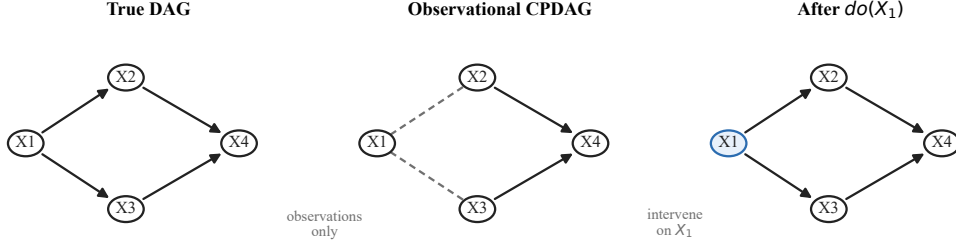


Figure 1: Observation identifies a CPDAG, not necessarily the full DAG. In this example the collider into X_4 is compelled from observational structure, while the two incident edges at X_1 remain undirected until an intervention on X_1 resolves them.

Andersson et al., 1997]. ACDB represents this observational ceiling by the completed partially directed acyclic graph (CPDAG) of the instance. Directed CPDAG edges are compelled by the observational distribution, while undirected edges mark orientations that remain unresolved from $P(X)$ alone. The active task is therefore structured around two different causal objects (Figure 1): the CPDAG is the correct observational target, while the final submission target is a fully oriented DAG.

Public runtime. The agent interacts with each instance through a strict observe-intervene-submit protocol. The runtime releases the fixed observational sample $D^{\text{obs}} \in \mathbb{R}^{n_{\text{obs}} \times d}$ exactly once. After observation, the agent may spend a finite intervention budget on actions of the form `intervene(var, value)`, each of which returns a fresh sample matrix from the intervened SCM with shape $n_{\text{int}} \times d$. Interventions are single-node and budgeted one call at a time; they are illegal before observation and impossible after the session is sealed. The episode terminates at `submit_graph`, where the agent submits a mixed graph with directed and undirected edges. Allowing undirected submissions is deliberate: leaving an edge unresolved is a different outcome from omitting an adjacency or reversing a direction that should have been identified.

Accepted benchmark instance. One accepted instance packages the tuple $(G, \text{CPDAG}(G), \mathcal{S}, \mathcal{I}^*, D^{\text{obs}}, B, \pi)$, where $\mathcal{S}$ is the parameterized SCM, $\mathcal{I}^*$ is a benchmark-owned minimum intervention set that resolves the CPDAG ambiguity for that world, $B = |\mathcal{I}^*| + \text{budget_slack}$ is the public intervention budget, and π is a random permutation applied to anonymize node identities. Assembly is rejection sampling at three gates — on the graph profile, on near-faithfulness of the parameterized SCM, and on the resolving-intervention profile (Algorithm 1); predicate-level definitions of the gates are in Appendix A.3. The agent never sees G , $\text{CPDAG}(G)$, $\mathcal{S}$, or $\mathcal{I}^*$ directly: it sees only anonymized variable identities, the observational matrix, the remaining budget, and any interventional sample matrices returned by the runtime.

Benchmark family and assumptions. The reported benchmark family is generated from level-specific configurations that vary graph size, edge count, observational and interventional sample counts, noise variance, and intervention slack while keeping the public interface fixed across levels. ACDB assumes full observability, causal sufficiency, and a linear-Gaussian SCM family. It also filters out near-unfaithful instances [Spirtes et al., 2000], so accepted worlds do not sit arbitrarily close to vanishing partial correlations that would blur the intended observational structure at finite samples. Under these assumptions, classical constraint-based discovery is meaningful: with an exact independence oracle and enough data, PC recovers the true CPDAG, so finite-sample deviations can be interpreted as observational recovery error rather than latent-confounding error [Spirtes et al., 2000]. This is the benchmark setting within which the later score layers and ablations should be read.

Algorithm 1 Accepted benchmark instance assembly. One iteration of the rejection loop draws a DAG, parameterizes a linear-Gaussian SCM, computes the resolving-intervention set, then permutes labels and samples the observational data. Three reject gates discard graphs with trivial CPDAGs, near-unfaithful SCMs, and CPDAGs that are already fully oriented. Predicate definitions are in Appendix A.3.

Require: level config ($d, k, \text{weight_range}, \text{noise_var}, \text{faithfulness_eps}, n_{\text{obs}}, \text{budget_slack}$);
 rng
Ensure: accepted instance ($G, \text{CPDAG}(G), \mathcal{S}, \mathcal{I}^*, D^{\text{obs}}, B, \pi$)

```

1: repeat
2:    $G \leftarrow \text{sample\_random\_dag}(d, k; \text{rng})$ 
3:    $C \leftarrow \text{dag\_to\_cpdag}(G)$ 
4:   if  $\text{reject\_graph}(G, C)$  then continue
5:   end if
6:    $\mathcal{S} \leftarrow \text{parameterize\_linear\_gaussian\_scm}(G, \text{weight\_range}, \text{noise\_var}; \text{rng})$ 
7:   if  $\text{reject\_scm}(\mathcal{S}, \text{faithfulness\_eps})$  then continue
8:   end if
9:    $\mathcal{I}^* \leftarrow \text{compute\_minimum\_intervention\_set}(G, C)$ 
10:  if  $\text{reject\_intervention\_profile}(\mathcal{I}^*, C)$  then continue
11:  end if
12:   $\pi \leftarrow \text{random\_permutation}(d; \text{rng})$ 
13:  relabel ( $G, C, \mathcal{S}, \mathcal{I}^*$ ) under  $\pi$ 
14:   $D^{\text{obs}} \leftarrow \text{sample\_observational\_data}(\mathcal{S}, n_{\text{obs}}; \text{rng})$ 
15:   $B \leftarrow |\mathcal{I}^*| + \text{budget\_slack}$ 
16:  return ( $G, C, \mathcal{S}, \mathcal{I}^*, D^{\text{obs}}, B, \pi$ )
17: until accepted

```

121 3 Scoring and Calibration

122 Section 2 defined the accepted instance family and the public observe-intervene-submit runtime. This
 123 section defines how submissions over those instances are read. In ACDB, the same submitted graph
 124 is evaluated against two retained truth objects, the instance CPDAG and the hidden DAG, and the
 125 reported ladder is calibrated so directed-edge scores are not interpreted in isolation (Figure 2).

126 3.1 Layered scoring contract

127 Each accepted instance retains two different truth objects. The first is the instance CPDAG, which
 128 represents the strongest structure justified by observation alone. The second is the hidden DAG, which
 129 remains the final target once interventions have been used. ACDB therefore separates observationally
 130 justified structure from final directed recovery instead of collapsing every graph error into a single
 131 score.

132 Against the true CPDAG, ACDB computes skeleton F1 on the CPDAG skeleton and compelled-edge
 133 F1 on the directed part of the CPDAG. These scores ask whether the submission recovered the
 134 observationally identifiable structure of the instance. A reversed compelled edge is counted as both a
 135 false positive on the submitted direction and a false negative on the true one, so adjacency preservation
 136 alone does not hide an observational orientation error.

137 Against the true DAG, ACDB computes directed-edge precision, recall, and F1 on submitted directed
 138 edges, together with structural Hamming distance (SHD). SHD assigns one error to each extra,
 139 missing, reversed, or unresolved true edge, where unresolved means that the true edge is directed
 140 but the submission leaves the same pair undirected. This distinction matters in ACDB because an
 141 undirected submitted edge is an explicit claim that the adjacency is present but the final orientation
 142 was not resolved.

143 The DAG layer is intentionally not Markov-equivalence-aware. Observational ambiguity is already
 144 represented by the CPDAG-facing scores, so giving additional partial credit at the DAG layer would
 145 blur the distinction between what the agent could justify from observation and what it ultimately
 146 resolved through active interaction.

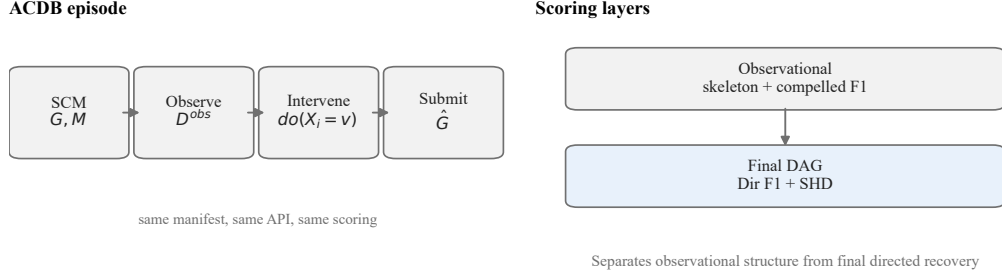


Figure 2: ACDB fixes the episode protocol and separates adjacency, orientation, and budget-use scores.

3.2 Difficulty ladder

The main paper reports results on five benchmark levels, L1-L5. These levels define the reported benchmark family. Every policy and model is run on the same paired accepted-seed manifest, so cross-method comparisons are instance matched rather than averaged over different sampled worlds.

Across these levels, the public interface is fixed while the accepted instance family changes along the main structural axes of the task: graph size, edge count, graph density, sample support, and intervention slack. The ladder therefore determines which causal worlds are compared under the same runtime. Because the accepted-seed manifest is shared across methods, later differences can be attributed to the compared policies and models rather than to different sampled instances.

3.3 Random-floor calibration

Directed-edge scores are only meaningful when read against a benchmark-owned floor. In ACDB, that floor is a structure-blind random submission rule that may use the public variable count d but not the hidden edge count k , the observational sample, or any intervention outcomes. Even under that restriction, dense DAG families can produce non-trivial directed F1 simply because many sampled directions land on true adjacencies. We refer to this benchmark-design hazard as density leakage.

Let

$$\rho = \frac{k}{\binom{d}{2}}. \quad (2)$$

Probe measurements across graph densities yield the heuristic fit

$$\mathbb{E}[\text{DirF1}_{\text{random}}] \approx \frac{\rho}{1 + 2\rho}. \quad (3)$$

Equation (3) is a probe-based approximation rather than a derived bound. Its role is practical: it identifies density regimes in which a blind submission rule can achieve misleadingly strong raw directed F1.

The reported L1-L5 ladder is calibrated with this effect in mind. All included levels stay below the tutorial-style density regime, and the resulting structure-blind floor remains materially lower than the high-density case. Directed-F1 differences should therefore be read relative to this floor rather than as absolute values detached from graph density. Appendix C derives the closed-form floor (Figure 4) and lists per-level values used in the §5 tables.

4 Experimental Setup

All methods are evaluated on the same accepted L1-L5 instances and scored by the same contract from Section 3. For each level, every compared policy is run on the same paired accepted-seed manifest, so later differences are measured on matched causal worlds rather than on different sampled instances. The comparisons in this section differ only in which part of the active-discovery pipeline is supplied, fixed, or left to the policy itself.

Table 1: Reported ACDB ladder levels. Each level fixes graph size d , true edge count k , density ρ , observational and interventional sample counts, noise variance, and intervention slack above the benchmark-owned minimum resolving set. The total public budget remains instance specific because it is defined relative to $|Z^*|$.

Level	d	k	ρ	n_{obs}	n_{int}	σ^2	slack
L1	6	6	0.400	50	25	1.0	2
L2	8	9	0.321	50	25	1.0	2
L3	10	12	0.267	50	25	1.0	2
L4	12	14	0.212	50	25	1.0	2
L5	14	16	0.176	50	25	1.0	2

178 4.1 Ablation design

179 The compared policy set separates three questions over the same benchmark family: how well a
 180 method constructs observational structure from D^{obs} ; how well it uses interventions once a reasonable
 181 observational graph is available; and how well it integrates both stages end to end. The hybrid policies
 182 make those cuts explicit. Per-policy allowed-action sets, prompts, and code paths are pinned down in
 183 Appendix D.6.

184 The classical reference policy, PC + greedy (pc_greedy), applies PC with Fisher- z conditional-
 185 independence tests at $\alpha = 0.05$ to the observational sample [Spirtes et al., 2000]. It then resolves
 186 the remaining undirected edges with a fixed intervention heuristic that intervenes on the highest-
 187 undirected-degree node and orients adjacent edges with a z -calibrated mean-shift rule (Appendix D.5).

188 The two end-to-end LLM policies keep the full active task exposed. LLM raw (llm_raw)
 189 receives the observational matrix, may intervene, and must submit the final mixed graph it-
 190 self. LLM stats (llm_stats) keeps that same burden but adds Fisher- z -style correlation,
 191 partial_correlation, and independence_test actions (Appendix D.2), so the comparison
 192 isolates whether explicit CI-style statistical primitives improve active discovery under the same
 193 runtime.

194 The two hybrid policies remove one stage of the pipeline at a time. PC CPDAG + LLM
 195 (pc_cpdag_llm) supplies the PC partial graph together with observational summary statistics,
 196 but withholds the raw observational matrix; what remains is the active use of interventions and
 197 the final orientation of the graph. LLM stats + greedy (llm_stats_cpdag_greedy) makes the
 198 complementary cut: the model uses the statistical interface to construct a partial graph and submit it,
 199 after which the same greedy intervention resolver completes the active stage.

200 4.2 Model panel

201 The paper evaluates a compact five-model panel: GPT-5.5, GPT-5.4-mini, Sonnet-4.6, Haiku-4.5,
 202 and Gemini-3-Flash. The panel is small by design: it is large enough to test whether the same
 203 decomposition remains visible across a meaningful capability spread, but not so large that the paper
 204 turns into a model ranking exercise.

205 4.3 Run protocol

206 The main paper reports only L1-L5. For each level, all policies and models are evaluated on the
 207 same accepted manifest of eight seeds (Appendix G.3), so all reported comparisons are paired at the
 208 instance level.

209 All LLM policies run in the same sequential single-action loop (Appendix D.7): at each step the
 210 model receives the current session state and the accumulated interaction history, then emits exactly
 211 one action from its policy-specific action set. Parallel tool use is disabled, and the final allowed step is
 212 submit-only. Raw-style policies use a maximum of 20 actions per episode, while stats-style policies
 213 use a maximum of 40.

5 Results

We report paired results on the L1-L5 benchmark family for the five active ablations from Section 4. The discussion moves from the average cross-model comparison to a selected layered read and then to behavior across the ladder.

5.1 Cross-model comparison on the average ladder

Figure 3 shows the main average comparison. On the end-to-end `llm_raw` condition, performance ranges from 0.697 directed F1 for GPT-5.5 down to 0.113 for Haiku-4.5, with Sonnet-4.6 at 0.314, Gemini-3-Flash at 0.281, and GPT-5.4-mini at 0.145. GPT-5.5 and Sonnet-4.6 lead on SHD as well (6.3 and 12.3), but the bottom three flip places under SHD — Haiku-4.5 (18.9) and Gemini-3-Flash (19.3) sit below GPT-5.4-mini (22.4) because the smaller models submit fewer edges and accumulate fewer extra-edge errors at low directed F1.

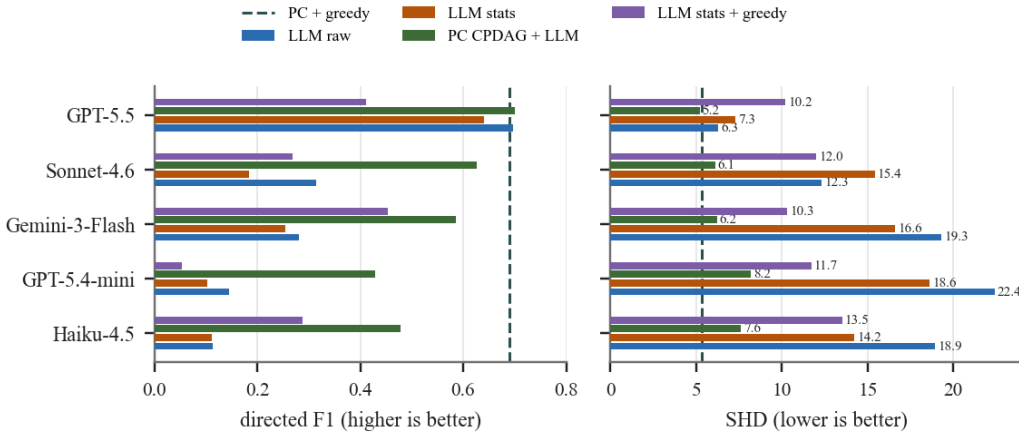


Figure 3: Only GPT-5.5 keeps pace with `pc_greedy` (dashed) on end-to-end `llm_raw`; the four weaker models close most of the gap once supplied with the PC-derived CPDAG (`pc_cpdag_llm`, green), while statistical-tool access alone (`llm_stats`, orange) sits below `llm_raw` for every model. Average over L1-L5.

The same figure also makes the ablation pattern visible. Relative to `llm_raw`, supplying the PC-derived CPDAG yields large directed-F1 gains for GPT-5.4-mini (+0.284), Sonnet-4.6 (+0.312), Haiku-4.5 (+0.366), and Gemini-3-Flash (+0.304), while the gain for GPT-5.5 is only +0.004. By contrast, `llm_stats` remains below `llm_raw` for all five models. Across the current ladder, the larger variation comes from model capability and from the structural-prior cut rather than from adding CI-style statistical actions alone. Full per-model average tables across all five layered metrics are in Appendix E.2.

5.2 Selected layered-score comparison

Table 2 gives one layered read over the strongest model. The end-to-end `llm_raw` policy is nearly tied with `pc_greedy` on directed F1 (0.697 vs 0.689), but the same comparison differs on both `comp_f1` (0.732 vs 0.612) and SHD (6.3 vs 5.3). The scores therefore separate different error profiles rather than collapsing them into one ranking.

The same slice also clarifies the stronger ablation. `pc_cpdag_llm` slightly improves both directed F1 and SHD relative to `pc_greedy`, while keeping the CPDAG-facing scores close. In this setting, supplying the observational structure changes the comparison from end-to-end discovery to final orientation quality. That is the narrowest place in the paper where the strongest model comes close to the classical active baseline, and it is best read as a secondary observation rather than the headline. The same layered breakdown for the other four models is in Appendix E.2.

Table 2: Selected GPT-5.5 average L1-L5 results. Higher is better for `skel_f1`, `comp_f1`, and `dir_f1`; lower is better for SHD.

Policy	<code>skel_f1</code>	<code>comp_f1</code>	<code>dir_f1</code>	SHD
<code>pc_greedy</code>	0.780	0.612	0.689	5.3
<code>pc_cpdag_llm</code>	0.789	0.615	0.701	5.2
<code>llm_raw</code>	0.753	0.732	0.697	6.3
<code>llm_stats</code>	0.704	0.673	0.640	7.3

5.3 Behavior across the ladder

The gap also widens as the accepted instances become harder. The classical `pc_greedy` baseline drops from 0.802 at L1 to 0.642 at L5 (−0.160). Over the same range, `llm_raw` drops from 0.824 to 0.537 for GPT-5.5 (−0.287), from 0.426 to 0.252 for Sonnet-4.6 (−0.174), from 0.414 to 0.185 for Gemini-3-Flash (−0.229), and from 0.192 to 0.047 for Haiku-4.5 (−0.145, but starting near the floor with little room to fall further). On the reported L1-L5 family, the four `llm_raw` runs that start at non-trivial directed F1 at L1 therefore lose more across the ladder than the classical active baseline does, while Haiku-4.5 sits near the floor throughout. The full per-level $\times$ per-model breakdown across all policies is in Appendix E.3, and trace-level views of representative successes and failures behind these averages are in Appendix F.

6 Related Work

LLMs for causal reasoning and graph discovery. Recent work has used language models for structured causal-discovery behavior rather than only pairwise causal judgments. Lampinen et al. [2023] study whether agents and language models can acquire active causal strategies from passive experience; Jiralerspong et al. [2024] use LLMs to propose full causal graphs through a breadth-first query schedule; Abdulaal et al. [2024] combine metadata- and data-driven reasoning to recover causal structure; and Vashishtha et al. [2023] use LLM-derived causal orders as a discovery prior for downstream effect estimation. These are method neighbors to ACDB: they treat causal structure recovery as an LLM-facing problem, but they do not provide a shared benchmark that places end-to-end agents, hybrid policies, and classical active baselines under the same intervention protocol and scoring contract.

Benchmarks for LLM causal discovery. Existing benchmark work splits between static causal reasoning and interactive graph discovery. Jin et al. [2024] tests whether models can infer causal direction from correlational statements alone, and Chen et al. [2025] evaluates iterative causal-graph identification through interventions and is the closest benchmark competitor to ACDB. Havrilla et al. [2025] studies the complementary setting in which an agent uses semantic metadata and edge experiments rather than anonymized observational and interventional samples. Srivastava et al. [2025] argues that current LLM causal-reasoning benchmarks are vulnerable to pretraining contamination and calls for science-grounded, novel evaluation protocols. ACDB sits closest to this benchmark line: it keeps the task synthetic and controlled, anonymizes variable identities to block semantic retrieval, exposes a fixed observe–intervene–submit runtime, and scores submitted graphs with a layered contract rather than a single success criterion.

7 Limitations

ACDB’s instance family is fully observed, causally sufficient, linear–Gaussian, and restricted to hard single-node interventions; this is the regime in which CPDAG identifiability, the resolving-intervention set, and the closed-form random floor are exactly defined, and the reported ladder caps graph size at $d = 14$. The empirical study covers a five-model, five-policy panel evaluated through proprietary APIs at eight paired seeds per level, reports point estimates rather than confidence intervals, and evaluates prompt-only inference. Each of these axes — non-linear and partially observed substrates, soft and multi-node interventions, larger graphs, denser panels, and fine-tuned or RL-trained agents — is a natural extension that inherits ACDB’s scoring discipline.

8 Conclusion

ACDB is a measurement instrument first and an empirical study second. Its design choices — two retained truth objects driving a layered scoring contract, a random-floor-calibrated ladder, paired accepted-seed manifests across methods, and hybrid ablations that cut the pipeline at the joints SCM theory already identifies — are what make differences between methods attributable rather than merely measured. The §5 patterns illustrate the point: that statistical-tool access does not reliably improve directed recovery, that supplying the PC-derived CPDAG closes most of the gap for weaker models but barely moves GPT-5, and that end-to-end LLM policies degrade faster than `pc_greedy` with graph scale, are interpretable as separable claims about observational recovery, interventional orientation, and end-to-end integration only because the instrument separates those stages. Without that separation, the same evidence would collapse into a single ranking and the same models would be called good or bad at "causal reasoning" with no further structure.

The fully observed linear-Gaussian regime with hard single-node interventions is the precondition for that separation. It is the setting in which every score layer has an exact theoretical referent: CPDAG identifiability is unambiguous, the resolving-intervention set is well-defined, and the structure-blind random floor admits a closed form. The substantive limitations this entails for non-linear, partially observed, or semantic substrates are stated explicitly in §7; widening the instrument to those regimes is scientifically valuable but reintroduces exactly the ambiguity the present version was built to remove. ACDB is therefore a base measurement layer, designed so that future extensions inherit its scoring discipline rather than replace it.

Treated as such, the instrument enables questions this paper does not yet answer. Whether a new model's gains on causal reasoning come from observational pattern matching, from intervention choice, or from sequential evidence integration becomes a question the layered scores and hybrid ablations can address directly. Whether tool augmentation or fine-tuning moves the residual orientation gap, or only the observational layer, becomes a controlled comparison rather than a debate. Whether the LLM-vs-classical degradation gap closes at larger graph scales for any model family becomes a measurable curve. ACDB is built so that the conclusions it supports remain legible as both the scientific environments and the agents become more capable.

References

- Ahmed Abdulaal, Adamos Hadjivasiliou, Nina Montaña-Brown, Tiantian He, Ayodeji Ijishakin, Ivana Drobnjak, Daniel C. Castro, and Daniel C. Alexander. Causal modelling agents: Causal graph discovery through synergising metadata- and data-driven reasoning. In *International Conference on Learning Representations (ICLR)*, 2024.
- Steen A. Andersson, David Madigan, and Michael D. Perlman. A characterization of markov equivalence classes for acyclic digraphs. *Annals of Statistics*, 25(2):505–541, 1997.
- Tingting Chen, Srinivas Anumasa, Beibei Lin, Vedant Shah, Anirudh Goyal, and Dianbo Liu. Auto-Bench: An automated benchmark for scientific discovery in LLMs. *arXiv preprint arXiv:2502.15224*, 2025.
- Frederick Eberhardt, Clark Glymour, and Richard Scheines. On the number of experiments sufficient and in the worst case necessary to identify all causal relations among N variables. In *Proceedings of the 21st Conference on Uncertainty in Artificial Intelligence (UAI)*, pages 178–184, 2005. arXiv:1207.1389.
- Alain Hauser and Peter Bühlmann. Characterization and greedy learning of interventional Markov equivalence classes of directed acyclic graphs. *Journal of Machine Learning Research*, 13:2409–2464, 2012.
- Alex Havrilla, David Alvarez-Melis, and Nicolo Fusi. IGDA: Interactive graph discovery through large language model agents. *arXiv preprint arXiv:2502.17189*, 2025.
- Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik R. Narasimhan. SWE-bench: Can language models resolve real-world GitHub issues? In *International Conference on Learning Representations (ICLR)*, 2024. arXiv:2310.06770.
- Zhijing Jin, Jiarui Liu, Zhiheng Lyu, Spencer Poff, Mrinmaya Sachan, Rada Mihalcea, Mona Diab, and Bernhard Schölkopf. Can large language models infer causation from correlation? In *International Conference on Learning Representations (ICLR)*, 2024. arXiv:2306.05836.
- Thomas Jiralerspong, Xiaoyin Chen, Yash More, Vedant Shah, and Yoshua Bengio. Efficient causal graph discovery using large language models. *arXiv preprint arXiv:2402.01207*, 2024.
- Emre Kıcıman, Robert Ness, Amit Sharma, and Chenhao Tan. Causal reasoning and large language models: Opening a new frontier for causality. *Transactions on Machine Learning Research (TMLR)*, 2023. arXiv:2305.00050.
- Andrew Kyle Lampinen, Stephanie C. Y. Chan, Ishita Dasgupta, Andrew J. Nam, and Jane X. Wang. Passive learning of active causal strategies in agents and language models. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023. arXiv:2305.16183.
- Xiao Liu, Hao Yu, Hanchen Zhang, Yifan Xu, Xuanyu Lei, Hanyu Lai, Yu Gu, et al. AgentBench: Evaluating LLMs as agents. In *International Conference on Learning Representations (ICLR)*, 2024. arXiv:2308.03688.
- Grégoire Mialon, Clémentine Fourrier, Craig Swift, Thomas Wolf, Yann LeCun, and Thomas Scialom. GAIA: A benchmark for general AI assistants. *arXiv preprint arXiv:2311.12983*, 2023.
- Peter Spirtes, Clark Glymour, and Richard Scheines. *Causation, Prediction, and Search*. Adaptive Computation and Machine Learning. MIT Press, 2nd edition, 2000.
- Ashutosh Srivastava, Lokesh Nagalapatti, Gautam Jajoo, Aniket Vashishtha, Parameswari Krishnamurthy, and Amit Sharma. Realizing LLMs’ causal potential requires science-grounded, novel benchmarks. *arXiv preprint arXiv:2510.16530*, 2025.
- Aniket Vashishtha, Abbavaram Gowtham Reddy, Abhinav Kumar, Saketh Bachu, Vineeth N. Balasubramanian, and Amit Sharma. Causal inference using LLM-guided discovery. *arXiv preprint arXiv:2310.15117*, 2023.
- Thomas S. Verma and Judea Pearl. Equivalence and synthesis of causal models. In *Proceedings of the 6th Conference on Uncertainty in Artificial Intelligence (UAI)*, pages 255–270, 1990.

- 360 Tianbao Xie et al. OSWorld: Benchmarking multimodal agents for open-ended tasks in real computer
 361 environments. In *Advances in Neural Information Processing Systems (NeurIPS), Datasets and*
 362 *Benchmarks Track*, 2024. arXiv:2404.07972.
- 363 Matej Zečević, Moritz Willig, Devendra Singh Dhami, and Kristian Kersting. Causal parrots: Large
 364 language models may talk causality but are not causal. *Transactions on Machine Learning Research*
 365 *(TMLR)*, 2023. arXiv:2308.13067.
- 366 Shuyan Zhou, Frank F. Xu, Hao Zhu, Xuhui Zhou, Robert Lo, Abishek Sridhar, Xianyi Cheng,
 367 Tianyue Ou, Yonatan Bisk, Daniel Fried, Uri Alon, and Graham Neubig. WebArena: A realistic
 368 web environment for building autonomous agents. In *International Conference on Learning*
 369 *Representations (ICLR)*, 2024. arXiv:2307.13854.

370 NeurIPS Paper Checklist

371 1. Claims

372 Question: Do the main claims made in the abstract and introduction accurately reflect the
 373 paper’s contributions and scope?

374 Answer: [\[Yes\]](#)

375 Justification: The abstract and §1 frame ACDB as a measurement instrument with a layered
 376 scoring contract and random-floor calibration; §2–§4 instantiate exactly those components
 377 and §5 reports the empirical patterns the abstract names (model spread, structural-prior
 378 gains, persistent end-to-end gap, no reliable lift from CI tools).

379 Guidelines:

- 380 • The answer [\[N/A\]](#) means that the abstract and introduction do not include the claims
 381 made in the paper.
- 382 • The abstract and/or introduction should clearly state the claims made, including the
 383 contributions made in the paper and important assumptions and limitations. A [\[No\]](#) or
 384 [\[N/A\]](#) answer to this question will not be perceived well by the reviewers.
- 385 • The claims made should match theoretical and experimental results, and reflect how
 386 much the results can be expected to generalize to other settings.
- 387 • It is fine to include aspirational goals as motivation as long as it is clear that these goals
 388 are not attained by the paper.

389 2. Limitations

390 Question: Does the paper discuss the limitations of the work performed by the authors?

391 Answer: [\[Yes\]](#)

392 Justification: §7 is a dedicated Limitations section that names the linear–Gaussian, full-
 393 observability, causal-sufficiency, and hard single-node intervention assumptions, the ladder’s
 394 $d \leq 14$ scale cap, the eight-paired-seed panel size and absence of confidence intervals,
 395 prompt-only inference through proprietary APIs (no fine-tuned or RL-trained agents), and
 396 the natural extensions that a future version inherits.

397 Guidelines:

- 398 • The answer [\[N/A\]](#) means that the paper has no limitation while the answer [\[No\]](#) means
 399 that the paper has limitations, but those are not discussed in the paper.
- 400 • The authors are encouraged to create a separate “Limitations” section in their paper.
- 401 • The paper should point out any strong assumptions and how robust the results are to
 402 violations of these assumptions (e.g., independence assumptions, noiseless settings,
 403 model well-specification, asymptotic approximations only holding locally). The authors
 404 should reflect on how these assumptions might be violated in practice and what the
 405 implications would be.
- 406 • The authors should reflect on the scope of the claims made, e.g., if the approach was
 407 only tested on a few datasets or with a few runs. In general, empirical results often
 408 depend on implicit assumptions, which should be articulated.

- The authors should reflect on the factors that influence the performance of the approach. For example, a facial recognition algorithm may perform poorly when image resolution is low or images are taken in low lighting. Or a speech-to-text system might not be used reliably to provide closed captions for online lectures because it fails to handle technical jargon.
- The authors should discuss the computational efficiency of the proposed algorithms and how they scale with dataset size.
- If applicable, the authors should discuss possible limitations of their approach to address problems of privacy and fairness.
- While the authors might fear that complete honesty about limitations might be used by reviewers as grounds for rejection, a worse outcome might be that reviewers discover limitations that aren't acknowledged in the paper. The authors should use their best judgment and recognize that individual actions in favor of transparency play an important role in developing norms that preserve the integrity of the community. Reviewers will be specifically instructed to not penalize honesty concerning limitations.

3. Theory assumptions and proofs

Question: For each theoretical result, does the paper provide the full set of assumptions and a complete (and correct) proof?

Answer: [N/A]

Justification: The paper makes no theorem-level claims; the only formal statements are the linear-Gaussian SCM definition (Eq. 1), the density definition (Eq. 2), and the random-floor approximation (Eq. 3), which is explicitly described as a probe-based heuristic rather than a derived bound, with the closed-form floor derivation deferred to Appendix C.

Guidelines:

- The answer [N/A] means that the paper does not include theoretical results.
- All the theorems, formulas, and proofs in the paper should be numbered and cross-referenced.
- All assumptions should be clearly stated or referenced in the statement of any theorems.
- The proofs can either appear in the main paper or the supplemental material, but if they appear in the supplemental material, the authors are encouraged to provide a short proof sketch to provide intuition.
- Inversely, any informal proof provided in the core of the paper should be complemented by formal proofs provided in appendix or supplemental material.
- Theorems and Lemmas that the proof relies upon should be properly referenced.

4. Experimental result reproducibility

Question: Does the paper fully disclose all the information needed to reproduce the main experimental results of the paper to the extent that it affects the main claims and/or conclusions of the paper (regardless of whether the code and data are provided or not)?

Answer: [Yes]

Justification: §2 (ladder Table 1, instance-assembly Algorithm 1), §4 (per-policy interfaces and budgets), and Appendix G (launch command, model IDs and events.jsonl paths, accepted-seed manifest, environment, repository layout) together specify the assembly, scoring, policies, models, seeds, and exact event-log paths from which every reported number was aggregated.

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- If the paper includes experiments, a [No] answer to this question will not be perceived well by the reviewers: Making the paper reproducible is important, regardless of whether the code and data are provided or not.
- If the contribution is a dataset and/or model, the authors should describe the steps taken to make their results reproducible or verifiable.

- Depending on the contribution, reproducibility can be accomplished in various ways. For example, if the contribution is a novel architecture, describing the architecture fully might suffice, or if the contribution is a specific model and empirical evaluation, it may be necessary to either make it possible for others to replicate the model with the same dataset, or provide access to the model. In general, releasing code and data is often one good way to accomplish this, but reproducibility can also be provided via detailed instructions for how to replicate the results, access to a hosted model (e.g., in the case of a large language model), releasing of a model checkpoint, or other means that are appropriate to the research performed.
- While NeurIPS does not require releasing code, the conference does require all submissions to provide some reasonable avenue for reproducibility, which may depend on the nature of the contribution. For example
 - (a) If the contribution is primarily a new algorithm, the paper should make it clear how to reproduce that algorithm.
 - (b) If the contribution is primarily a new model architecture, the paper should describe the architecture clearly and fully.
 - (c) If the contribution is a new model (e.g., a large language model), then there should either be a way to access this model for reproducing the results or a way to reproduce the model (e.g., with an open-source dataset or instructions for how to construct the dataset).
 - (d) We recognize that reproducibility may be tricky in some cases, in which case authors are welcome to describe the particular way they provide for reproducibility. In the case of closed-source models, it may be that access to the model is limited in some way (e.g., to registered users), but it should be possible for other researchers to have some path to reproducing or verifying the results.

5. Open access to data and code

Question: Does the paper provide open access to the data and code, with sufficient instructions to faithfully reproduce the main experimental results, as described in supplemental material?

Answer: [Yes]

Justification: The benchmark is the contribution: Appendix G.5 links to the anonymous repository containing the world generator, scoring, ablation policies, and trace-extraction scripts; Appendix G.1 gives the exact `uv run` command and Appendix G.2 lists per-model `events.jsonl` paths used by every reported number.

Guidelines:

- The answer [N/A] means that paper does not include experiments requiring code.
- Please see the NeurIPS code and data submission guidelines (<https://neurips.cc/public/guides/CodeSubmissionPolicy>) for more details.
- While we encourage the release of code and data, we understand that this might not be possible, so [No] is an acceptable answer. Papers cannot be rejected simply for not including code, unless this is central to the contribution (e.g., for a new open-source benchmark).
- The instructions should contain the exact command and environment needed to run to reproduce the results. See the NeurIPS code and data submission guidelines (<https://neurips.cc/public/guides/CodeSubmissionPolicy>) for more details.
- The authors should provide instructions on data access and preparation, including how to access the raw data, preprocessed data, intermediate data, and generated data, etc.
- The authors should provide scripts to reproduce all experimental results for the new proposed method and baselines. If only a subset of experiments are reproducible, they should state which ones are omitted from the script and why.
- At submission time, to preserve anonymity, the authors should release anonymized versions (if applicable).
- Providing as much information as possible in supplemental material (appended to the paper) is recommended, but including URLs to data and code is permitted.

6. Experimental setting/details

Question: Does the paper specify all the training and test details (e.g., data splits, hyperparameters, how they were chosen, type of optimizer) necessary to understand the results?

Answer: [Yes]

Justification: §2 fixes the per-level instance configuration (graph size, edge count, sample counts, noise variance, intervention slack); §4 fixes per-policy action sets, action budgets, and the sequential single-action loop; Appendix D.6 pins per-policy prompts and code paths and Appendix G.2 fixes per-model decoding settings and retry policy.

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- The experimental setting should be presented in the core of the paper to a level of detail that is necessary to appreciate the results and make sense of them.
- The full details can be provided either with the code, in appendix, or as supplemental material.

7. Experiment statistical significance

Question: Does the paper report error bars suitably and correctly defined or other appropriate information about the statistical significance of the experiments?

Answer: [No]

Justification: §5 reports paired-seed point estimates over the eight-seed accepted manifest per level (Appendix G.3); we did not include error bars because the paired-seed instance match is the intended unit of comparison and adding per-cell intervals at $n = 8$ would not change the ablation patterns reported, but this is acknowledged as a limitation in §7.

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- The authors should answer [Yes] if the results are accompanied by error bars, confidence intervals, or statistical significance tests, at least for the experiments that support the main claims of the paper.
- The factors of variability that the error bars are capturing should be clearly stated (for example, train/test split, initialization, random drawing of some parameter, or overall run with given experimental conditions).
- The method for calculating the error bars should be explained (closed form formula, call to a library function, bootstrap, etc.)
- The assumptions made should be given (e.g., Normally distributed errors).
- It should be clear whether the error bar is the standard deviation or the standard error of the mean.
- It is OK to report 1-sigma error bars, but one should state it. The authors should preferably report a 2-sigma error bar than state that they have a 96% CI, if the hypothesis of Normality of errors is not verified.
- For asymmetric distributions, the authors should be careful not to show in tables or figures symmetric error bars that would yield results that are out of range (e.g., negative error rates).
- If error bars are reported in tables or plots, the authors should explain in the text how they were calculated and reference the corresponding figures or tables in the text.

8. Experiments compute resources

Question: For each experiment, does the paper provide sufficient information on the computer resources (type of compute workers, memory, time of execution) needed to reproduce the experiments?

Answer: [Yes]

Justification: Appendix E.5 reports per-cell wallclock and per-call cost summaries for every (model, policy, level) cell behind the §5 numbers; runs were API-bound (no local GPU) on commodity hardware, with all retries logged in the per-model events.jsonl files cited in Appendix G.2.

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- The paper should indicate the type of compute workers CPU or GPU, internal cluster, or cloud provider, including relevant memory and storage.
- The paper should provide the amount of compute required for each of the individual experimental runs as well as estimate the total compute.
- The paper should disclose whether the full research project required more compute than the experiments reported in the paper (e.g., preliminary or failed experiments that didn't make it into the paper).

9. Code of ethics

Question: Does the research conducted in the paper conform, in every respect, with the NeurIPS Code of Ethics <https://neurips.cc/public/EthicsGuidelines>?

Answer: [Yes]

Justification: The benchmark uses synthetic linear-Gaussian SCM instances with anonymized variable identities; no human subjects, scraped data, or identifying information are involved, and all evaluated models are accessed through their official APIs under their published terms.

Guidelines:

- The answer [N/A] means that the authors have not reviewed the NeurIPS Code of Ethics.
- If the authors answer [No], they should explain the special circumstances that require a deviation from the Code of Ethics.
- The authors should make sure to preserve anonymity (e.g., if there is a special consideration due to laws or regulations in their jurisdiction).

10. Broader impacts

Question: Does the paper discuss both potential positive societal impacts and negative societal impacts of the work performed?

Answer: [N/A]

Justification: ACDB is an evaluation instrument for an upstream methodological capability (active causal discovery on synthetic SCMs) with no direct deployment surface; it does not train or release a model and does not target any application domain in which it could create direct positive or negative societal impact.

Guidelines:

- The answer [N/A] means that there is no societal impact of the work performed.
- If the authors answer [N/A] or [No], they should explain why their work has no societal impact or why the paper does not address societal impact.
- Examples of negative societal impacts include potential malicious or unintended uses (e.g., disinformation, generating fake profiles, surveillance), fairness considerations (e.g., deployment of technologies that could make decisions that unfairly impact specific groups), privacy considerations, and security considerations.
- The conference expects that many papers will be foundational research and not tied to particular applications, let alone deployments. However, if there is a direct path to any negative applications, the authors should point it out. For example, it is legitimate to point out that an improvement in the quality of generative models could be used to generate Deepfakes for disinformation. On the other hand, it is not needed to point out that a generic algorithm for optimizing neural networks could enable people to train models that generate Deepfakes faster.
- The authors should consider possible harms that could arise when the technology is being used as intended and functioning correctly, harms that could arise when the technology is being used as intended but gives incorrect results, and harms following from (intentional or unintentional) misuse of the technology.
- If there are negative societal impacts, the authors could also discuss possible mitigation strategies (e.g., gated release of models, providing defenses in addition to attacks, mechanisms for monitoring misuse, mechanisms to monitor how a system learns from feedback over time, improving the efficiency and accessibility of ML).

11. Safeguards

Question: Does the paper describe safeguards that have been put in place for responsible release of data or models that have a high risk for misuse (e.g., pre-trained language models, image generators, or scraped datasets)?

Answer: [N/A]

Justification: The released artifacts are a synthetic benchmark generator, scoring code, and event-log analysis scripts; there is no model release and the synthetic SCM data carries no misuse risk requiring access control.

Guidelines:

- The answer [N/A] means that the paper poses no such risks.
- Released models that have a high risk for misuse or dual-use should be released with necessary safeguards to allow for controlled use of the model, for example by requiring that users adhere to usage guidelines or restrictions to access the model or implementing safety filters.
- Datasets that have been scraped from the Internet could pose safety risks. The authors should describe how they avoided releasing unsafe images.
- We recognize that providing effective safeguards is challenging, and many papers do not require this, but we encourage authors to take this into account and make a best faith effort.

12. Licenses for existing assets

Question: Are the creators or original owners of assets (e.g., code, data, models), used in the paper, properly credited and are the license and terms of use explicitly mentioned and properly respected?

Answer: [Yes]

Justification: The classical baseline relies on the PC algorithm [Spirtes et al., 2000], cited at every use; LLM models are accessed via their providers' APIs under provider terms; software dependencies (NumPy, SciPy, NetworkX, LiteLLM) are used under their permissive licenses; no scraped or third-party datasets are involved.

Guidelines:

- The answer [N/A] means that the paper does not use existing assets.
- The authors should cite the original paper that produced the code package or dataset.
- The authors should state which version of the asset is used and, if possible, include a URL.
- The name of the license (e.g., CC-BY 4.0) should be included for each asset.
- For scraped data from a particular source (e.g., website), the copyright and terms of service of that source should be provided.
- If assets are released, the license, copyright information, and terms of use in the package should be provided. For popular datasets, `paperswithcode.com/datasets` has curated licenses for some datasets. Their licensing guide can help determine the license of a dataset.
- For existing datasets that are re-packaged, both the original license and the license of the derived asset (if it has changed) should be provided.
- If this information is not available online, the authors are encouraged to reach out to the asset's creators.

13. New assets

Question: Are new assets introduced in the paper well documented and is the documentation provided alongside the assets?

Answer: [Yes]

Justification: The benchmark itself is the new asset: §2 documents the instance schema, §3 the scoring contract, §4 the policy set, Appendix A.3 the predicate-level rejection gates, Appendix D.6 per-policy interfaces and prompts, and Appendix G.5 the repository layout and launch commands.

Guidelines:

- The answer [N/A] means that the paper does not release new assets.
- Researchers should communicate the details of the dataset/code/model as part of their submissions via structured templates. This includes details about training, license, limitations, etc.
- The paper should discuss whether and how consent was obtained from people whose asset is used.
- At submission time, remember to anonymize your assets (if applicable). You can either create an anonymized URL or include an anonymized zip file.

14. Crowdsourcing and research with human subjects

Question: For crowdsourcing experiments and research with human subjects, does the paper include the full text of instructions given to participants and screenshots, if applicable, as well as details about compensation (if any)?

Answer: [N/A]

Justification: No human subjects, crowdsourcing, or annotation were involved; all data are synthetic SCM samples and all agent traces come from API calls to the five evaluated models.

Guidelines:

- The answer [N/A] means that the paper does not involve crowdsourcing nor research with human subjects.
- Including this information in the supplemental material is fine, but if the main contribution of the paper involves human subjects, then as much detail as possible should be included in the main paper.
- According to the NeurIPS Code of Ethics, workers involved in data collection, curation, or other labor should be paid at least the minimum wage in the country of the data collector.

15. Institutional review board (IRB) approvals or equivalent for research with human subjects

Question: Does the paper describe potential risks incurred by study participants, whether such risks were disclosed to the subjects, and whether Institutional Review Board (IRB) approvals (or an equivalent approval/review based on the requirements of your country or institution) were obtained?

Answer: [N/A]

Justification: The research involves no human subjects and therefore no IRB review was required.

Guidelines:

- The answer [N/A] means that the paper does not involve crowdsourcing nor research with human subjects.
- Depending on the country in which research is conducted, IRB approval (or equivalent) may be required for any human subjects research. If you obtained IRB approval, you should clearly state this in the paper.
- We recognize that the procedures for this may vary significantly between institutions and locations, and we expect authors to adhere to the NeurIPS Code of Ethics and the guidelines for their institution.
- For initial submissions, do not include any information that would break anonymity (if applicable), such as the institution conducting the review.

16. Declaration of LLM usage

Question: Does the paper describe the usage of LLMs if it is an important, original, or non-standard component of the core methods in this research? Note that if the LLM is used only for writing, editing, or formatting purposes and does *not* impact the core methodology, scientific rigor, or originality of the research, declaration is not required.

Answer: [Yes]

Justification: LLMs appear in this paper exclusively as the evaluation subjects of the benchmark, not as a method component; their role as agents under test is described throughout §4 and §5, and per-model API configurations are pinned in Appendix G.2.

Guidelines:

- The answer [N/A] means that the core method development in this research does not involve LLMs as any important, original, or non-standard components.
- Please refer to our LLM policy in the NeurIPS handbook for what should or should not be described.

A Instance generation

This appendix formalises the accepted-instance family informally introduced in Section 2. The full assembly procedure lives in `src/causal_discovery/benchmark/instance.py`; the per-level configuration is fixed by the ladder in `run_ladder.py`.

A.1 DAG sampling

Each instance starts from a random topological order σ over d node ids, drawn uniformly from S_d . The sampler enumerates the $\binom{d}{2}$ forward pairs (i, j) with $\sigma^{-1}(i) < \sigma^{-1}(j)$ and selects exactly k of them uniformly at random without replacement to form the edge set. The sampling is uniform on the set of DAGs with exactly k forward edges under a uniformly random topological order; it is not Erdős–Rényi.

The five in-paper levels fix (d, k) as follows.

	L1	L2	L3	L4	L5
Nodes d	6	8	10	12	14
Edges k	6	9	12	14	16
Edge ratio k/d	1.00	1.13	1.20	1.17	1.14

A.2 Linear–Gaussian SCM parameterisation

For each edge $i \rightarrow j$ in the sampled DAG, the structural coefficient β_{ji} is drawn uniformly from $[-2.0, -0.5] \cup [0.5, 2.0]$, with the sign chosen by an independent fair coin. Edges absent from the DAG receive coefficient zero. Each node receives an independent Gaussian noise term $\varepsilon_j \sim \mathcal{N}(0, \sigma_j^2)$ with intercept zero. For all in-paper levels, $\sigma_j^2 = 1.0$ for every node (the $\sigma^2 = 0.5$ setting in the L0 spec is not reported).

The structural equations are

$$x_j = \sum_{i \in \text{pa}(j)} \beta_{ji} x_i + \varepsilon_j, \quad j = 1, \dots, d. \quad (4)$$

A.3 Rejection filters

Three filters are applied in sequence. A candidate is rejected and resampled if any filter fires.

Graph filter. The CPDAG of the sampled DAG must contain at least two undirected edges, those undirected edges must not all share a single common node, and the DAG skeleton must be connected. This excludes instances whose observational equivalence class trivially identifies the DAG and instances whose skeleton splits into disjoint components.

SCM filter. The implied covariance matrix must be non-singular, and every *relevant structural partial correlation* must satisfy $|\rho| \geq \varepsilon_{\text{faith}}$ with $\varepsilon_{\text{faith}} = 0.1$. The second condition is a finite-sample faithfulness margin: it removes parameter draws whose dependencies are too weak for any reasonable conditional-independence test to detect at the available sample sizes.

Intervention-profile filter. The minimum sufficient intervention set for the CPDAG (the lexicographically first set of single-node interventions that orient every undirected edge under Meek closure)

must be non-empty. An empty intervention set means the observational CPDAG already pins down the full DAG, which collapses the active-discovery loop.

A.4 Anonymisation and accepted-instance tuple

A surviving candidate is anonymised by drawing a uniform random permutation $\pi \in S_d$ and relabelling the DAG, the CPDAG, the SCM, and the intervention set under π . The agent observes the public node ids only; the inverse π^{-1} is retained by the evaluator and never exposed.

The accepted-instance tuple is

$$\mathcal{I} = (G_\pi, \mathcal{C}_\pi, \mathcal{M}_\pi, I_\pi^*, D_{\text{obs}}, B, \pi), \quad (5)$$

where G_π is the relabelled true DAG, $\mathcal{C}_\pi$ its CPDAG, $\mathcal{M}_\pi$ the relabelled linear-Gaussian SCM, I_π^* the relabelled minimum sufficient intervention set, $D_{\text{obs}} \in \mathbb{R}^{n_{\text{obs}} \times d}$ a fresh observational sample drawn from $\mathcal{M}_\pi$, B the per-episode intervention budget, and π the anonymisation permutation. The runtime exposes D_{obs} and B to the agent at episode start; everything else is hidden until scoring.

A.5 Budget semantics

The intervention budget is

$$B = |I_\pi^*| + s, \quad (6)$$

where s is the per-level slack. All in-paper levels use $s = 2$, so the agent always has at least two interventions in excess of the minimum sufficient set. Each intervention call to the runtime draws $n_{\text{int}} = 25$ samples from $\mathcal{M}_\pi$ under $\text{do}(V_j = v)$ and decrements the remaining budget by one.

B Scoring contract

This appendix gives the formal definitions of the metrics introduced in Section 3. The scoring code lives in `src/causal_discovery/scoring/scores.py`; the submission object is defined in `src/causal_discovery/scoring/submission.py`.

A submission is a partially directed graph $\mathcal{S} = (V_d, E_d, E_u)$ over the public node set, where E_d is a directed edge set whose directed subgraph is itself a DAG and E_u is a set of canonical undirected edges (a, b) with $a < b$. The two edge sets are disjoint: a pair cannot be both directed and unresolved. Self-loops and 2-cycles are rejected at submission time. The skeleton of $\mathcal{S}$ is

$$\text{skel}(\mathcal{S}) = \pi(E_d) \cup E_u, \quad (7)$$

where π projects each directed edge to its canonical unordered pair. All scores below operate on the public-labelled objects; the anonymisation permutation of Appendix A.4 is applied symmetrically to truth and submission, so it cancels.

For a TP/FP/FN triple, precision, recall, and F1 follow the standard definitions

$$P = \frac{TP}{TP + FP}, \quad R = \frac{TP}{TP + FN}, \quad F1 = \frac{2P \cdot R}{P + R}, \quad (8)$$

with the convention that $P = R = 1$ whenever the relevant denominator is zero, and $F1 = 0$ if $P + R = 0$.

B.1 Skeleton F1

Skeleton F1 measures adjacency recovery against the observational ceiling. Let $\text{skel}(\mathcal{C}_\pi)$ denote the skeleton of the true CPDAG (equivalently, the skeleton of the hidden DAG). The submitted skeleton is $\text{skel}(\mathcal{S})$ defined above. Counts are over unordered pairs:

$$TP = |\text{skel}(\mathcal{S}) \cap \text{skel}(\mathcal{C}_\pi)|, \quad FP = |\text{skel}(\mathcal{S}) \setminus \text{skel}(\mathcal{C}_\pi)|, \quad FN = |\text{skel}(\mathcal{C}_\pi) \setminus \text{skel}(\mathcal{S})|. \quad (9)$$

Skeleton F1 is symmetric in directed and unresolved submitted edges: both project to the same skeleton.

800 B.2 Compelled-edge F1

801 Let $E^c = E_d(\mathcal{C}_\pi)$ be the compelled edges of the true CPDAG (its directed portion). Compelled-edge
802 scoring iterates only over E^c . For each $(u, v) \in E^c$:

$$\begin{aligned} \text{if } (u, v) \in E_d(\mathcal{S}) &\Rightarrow \text{TP} += 1, \\ \text{else if } (v, u) \in E_d(\mathcal{S}) &\Rightarrow \text{FP} += 1 \text{ and } \text{FN} += 1, \\ \text{else} &\Rightarrow \text{FN} += 1. \end{aligned} \quad (10)$$

803 Two consequences are worth flagging. First, a reversed compelled edge is penalised twice, once
804 as a false positive and once as a false negative, because it both asserts an incorrect orientation and
805 fails to assert the correct one. Second, submitted directed edges that do not project to any pair in
806 $\pi(E^c)$ contribute nothing to the compelled-edge counts; they are scored elsewhere. This makes
807 compelled-edge F1 a directed measure restricted to the observationally identifiable orientations, not a
808 directed F1 evaluated on the CPDAG.¹

809 B.3 Directed F1

810 Directed F1 is the symmetric set-comparison of submitted directed edges against the hidden DAG:

$$\text{TP} = |E_d(\mathcal{S}) \cap E(G_\pi)|, \quad \text{FP} = |E_d(\mathcal{S}) \setminus E(G_\pi)|, \quad \text{FN} = |E(G_\pi) \setminus E_d(\mathcal{S})|. \quad (11)$$

811 A submitted undirected edge $\{u, v\} \in E_u$ contributes to none of these counts: it is not in $E_d(\mathcal{S})$, so
812 it cannot be a TP or FP, and it does not match any element of $E(G_\pi)$, so the corresponding true edge
813 falls into FN. Submitting unresolved orientations therefore reduces directed recall but never inflates
814 directed false positives.

815 B.4 Structural Hamming distance

816 SHD is computed over the union of true and submitted skeleton pairs,

$$\mathcal{P} = \pi(E(G_\pi)) \cup \text{skel}(\mathcal{S}). \quad (12)$$

817 For each $\{u, v\} \in \mathcal{P}$, exactly one of the following holds:

- 818 • **Extra:** $\{u, v\} \notin \pi(E(G_\pi))$ (the submission asserts an adjacency that is not in the true
819 DAG).
- 820 • **Missing:** $\{u, v\} \in \pi(E(G_\pi))$ and $\{u, v\} \notin \text{skel}(\mathcal{S})$.
- 821 • **Unresolved:** $\{u, v\} \in E_u$ and $\{u, v\} \in \pi(E(G_\pi))$.
- 822 • **Reversed:** $\{u, v\} \in \pi(E_d(\mathcal{S}))$ and $\{u, v\} \in \pi(E(G_\pi))$, but the submitted orientation is
823 opposite to the true one.
- 824 • **Correct:** $\{u, v\} \in \pi(E_d(\mathcal{S}))$, $\{u, v\} \in \pi(E(G_\pi))$, and the orientations match.

825 Each non-correct case contributes one error. The SHD is the total error count

$$\text{SHD}(\mathcal{S}, G_\pi) = n_{\text{extra}} + n_{\text{missing}} + n_{\text{unresolved}} + n_{\text{reversed}}, \quad (13)$$

826 which is the four-error decomposition reported alongside SHD in the per-model tables.

827 B.5 Worked example

828 Take the 4-node DAG G with edges $E(G) = \{1 \rightarrow 3, 2 \rightarrow 3, 1 \rightarrow 4\}$. Its CPDAG has the v-structure
829 $\{1 \rightarrow 3, 2 \rightarrow 3\}$ as compelled edges and the edge $\{1, 4\}$ undirected: Meek rule 1 cannot orient
830 $1 - 4$ because node 1 has no parents to propagate from, and rules 2 and 3 require directed edges
831 incident to node 4, of which there are none. Consider a submission $\mathcal{S}$ with directed edges $E_d(\mathcal{S}) =$
832 $\{(3, 1), (1, 4), (2, 4)\}$ and undirected edges $E_u(\mathcal{S}) = \{\{2, 3\}\}$. The metrics evaluate as follows.

¹When $E^c = \emptyset$ (the CPDAG has no compelled edges), the convention $P = R = 1$ applies and compelled-edge F1 equals 1 trivially. Such instances are excluded from the in-paper levels by construction (Appendix A.3: the rejection filters require at least two undirected edges and a non-empty minimum sufficient intervention set).

Metric	Counts	Value
Skeleton F1	TP = 3, FP = 1, FN = 0	F1 = 6/7 $\approx$ 0.857
Compelled-edge F1	TP = 0, FP = 1, FN = 2	F1 = 0
Directed F1	TP = 1, FP = 2, FN = 2	F1 = 1/3
SHD	1 reversed, 1 unresolved, 1 extra	SHD = 3

833 The compelled-edge edge (1, 3) is submitted reversed as (3, 1) and contributes one FP and one FN.
835 The compelled-edge edge (2, 3) is submitted as unresolved and contributes one FN, no FP. The
836 directed-F1 numerator credits only (1, 4), which appears in both $E_d(\mathcal{S})$ and $E(G)$. The SHD pair
837 (1, 3) is reversed, (2, 3) is unresolved, (2, 4) is extra, and (1, 4) is correct.

838 C Random-submission floor

839 This appendix calibrates the directed-F1 axis used in Section 5. Because directed F1 (Appendix B.3)
840 is bounded below by what a structure-blind submitter achieves rather than by zero, an absolute
841 score has no fixed meaning across levels: the ladder varies graph density and the floor varies with
842 it. C.1–C.2 give the sampler and its closed-form expected F1; C.3 confirms the closed form against
843 direct simulation; C.4 plots the curve with the v1 ladder range shaded; C.5 lists the per-level floors a
844 reader needs to subtract from a §5 number before drawing a conclusion.

845 C.1 Random-submission policy

846 The policy is the *uniform- m random DAG* sampler of `make_random_uniform_submission`
847 (`run_random_dag_baseline.py:200–217`). It uses only the public node count d ; it does not
848 call `observe()` or `intervene()`, does not consult the CPDAG or the true edge count k , and submits
849 no undirected edges. With $M = d(d - 1)/2$:

- 850 1. Draw a topological order $\pi \sim \text{Uniform}(S_d)$.
- 851 2. Draw $m \sim \text{Uniform}\{0, 1, \dots, M\}$.
- 852 3. Sample m edges uniformly without replacement from the M order-respecting candidate
853 pairs $\{(\pi(i), \pi(j)) : i < j\}$ and submit them as E_d , with $E_u = \emptyset$.

854 The resulting submission is a DAG by construction (it respects π); it is scored by the same
855 `score_submission` as any LLM or PC submission.

856 C.2 Closed-form expected directed F1

857 Let G be the true DAG with $|E(G)| = k$ edges. Consider one true directed edge $(u, v) \in E(G)$.
858 Under the sampler of C.1, the unordered pair $\{u, v\}$ appears among the m chosen pairs with
859 probability m/M by symmetry of uniform sampling without replacement. Conditional on inclusion,
860 the orientation in the submission is (u, v) iff π places u before v , which happens with probability
861 $1/2$ under uniform π . Therefore

$$\Pr[(u, v) \in E_d(\mathcal{S}) \mid m] = \frac{m}{2M}, \quad \mathbb{E}[|E_d(\mathcal{S}) \cap E(G)| \mid m] = \frac{km}{2M}. \quad (14)$$

862 With $|E_d(\mathcal{S})| = m$ and $|E(G)| = k$ deterministic given (m, k) , the per- m directed precision and
863 recall are $\mathbb{E}[\text{TP} \mid m] / m = k / (2M)$ and $\mathbb{E}[\text{TP} \mid m] / k = m / (2M)$, giving the harmonic mean

$$\mathbb{E}[F_1 \mid m, k] = \frac{km}{M(m + k)}. \quad (15)$$

864 Averaging uniformly over m yields the discrete closed form

$$\mathbb{E}[F_1 \mid d, k] = \frac{1}{M + 1} \sum_{m=0}^M \frac{km}{M(m + k)}, \quad (16)$$

865 implemented verbatim in `exact_expected_directed_f1` (`scripts/ladder_random_floor_sanity.py:65–72`).

866 C.3 Empirical confirmation

867 The probe draws $16 \times 100 = 1600$ uniform- m submissions per level, scores each
 868 through the canonical `score_submission`, and reports the per-level mean and standard error.
 869 The `v0_like_density_probe` rows of `traces/ladder_random_floor_sanity/summary.csv`
 870 cover L1–L4 at the production (d, k) ; the L5 entry is measured separately at the production
 871 $(d, k) = (14, 16)$ with the same protocol.

Level	(d, k)	ρ	$\mathbb{E}[F_1]$ (closed)	Measured $\pm$ SE	Gap
L1	(6, 6)	0.400	0.1957	0.1923 ± 0.0041	−0.0034
L2	(8, 9)	0.321	0.1734	0.1765 ± 0.0031	+0.0030
L3	(10, 12)	0.267	0.1547	0.1527 ± 0.0025	−0.0020
L4	(12, 14)	0.212	0.1330	0.1350 ± 0.0020	+0.0021
L5	(14, 16)	0.176	0.1166	0.1144 ± 0.0017	−0.0022

873 The gap is within 1.3 standard errors on every row, confirming Equation (16).

874 C.4 Floor curve

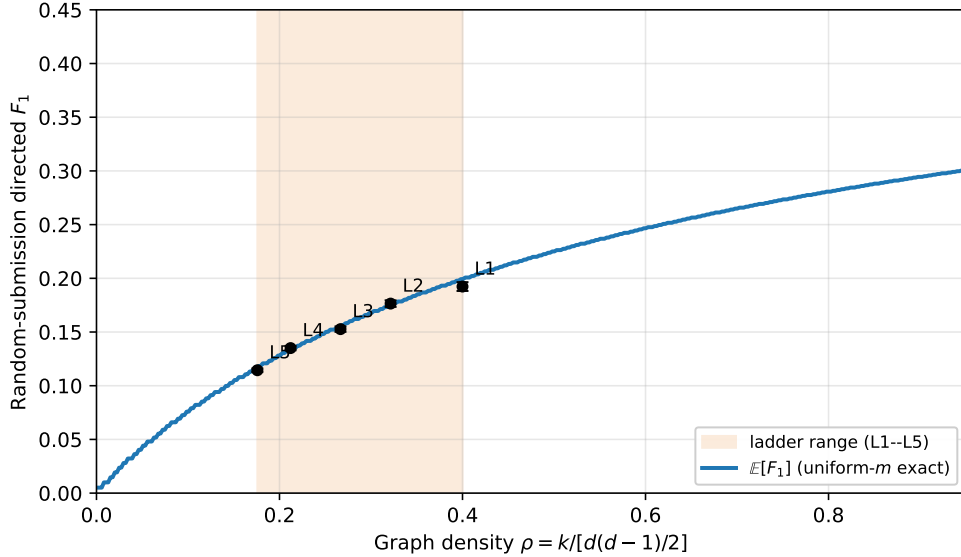


Figure 4: Random-submission directed-F1 floor as a function of graph density ρ . Solid: closed-form $\mathbb{E}[F_1]$ (Equation (16), evaluated at a reference $d = 20$; the curve is essentially flat in d once $d \geq 12$). Black dots with error bars: measured per-level means $\pm$ SE for L1–L5 from the probe of C.3. The shaded band marks the ladder density range; results in Section 5 should be read as gap above this floor at the level’s density, not as absolute F_1 .

875 C.5 Reading Section 5 against the floor

876 The per-level closed-form floors at the production (d, k) of `ladder_levels_v1`
 877 (`run_ladder.py:124-132`) are

L1: 0.196, L2: 0.173, L3: 0.155, L4: 0.133, L5: 0.117.

878 A model F1 of 0.30 is a +0.10 gap at L1 and a +0.18 gap at L5; the cross-level comparison that
 879 matters is the gap, not the absolute. We therefore report gap-above-floor alongside raw F1 in the
 880 per-model tables of Appendix E.

D Policy specifications

This appendix pins down the formal pieces that Section 4 summarises: the typed tool surface, the statistical primitives, the system prompts, the z -calibrated mean-shift rule, the per-policy allowed-action sets, and the runtime loop. The policy comparison table itself is not restated. Code paths refer to `run_ladder.py` and the modules under `src/causal_discovery/agents/`.

D.1 Tool surface

Each turn emits exactly one action via a strict JSON-schema tool call: the schema is generated by `make_action_response_schema(tool_schema.py)` with `strict: True` and `additionalProperties: False`, so every required field for the chosen action must be present, unused scalar fields are `null`, and unused list fields are `[]`. The five action signatures, lifted verbatim from `ACTION_FIELDS` and `FIELD_SCHEMAS`, are:

Action	Signature
<code>intervene</code>	<code>(var: int, value: float)</code>
<code>correlation</code>	<code>(i: int, j: int) -> float</code>
<code>partial_correlation</code>	<code>(i: int, j: int, conditioning_on: list[int]) -> float</code>
<code>independence_test</code>	<code>(i: int, j: int, conditioning_on: list[int], alpha: float) -> (bool, float)</code>
<code>submit_graph</code>	<code>(directed_edges: list[[int,int]], undirected_edges: list[[int,int]], reasoning_summary: str)</code>

Every action also carries a `reasoning_summary` string, which is appended to a working-memory deque (Appendix D.7).

D.2 Statistical primitives

The three CI-style tools are implemented in `stats_tools.py` and operate on the *active dataset*: the observational sample D^{obs} before any intervention, and the most recent intervention sample thereafter (`run_ladder.py`:578-579, 602-603, 629-630).

- `correlation(i, j)` returns the Pearson correlation $\hat{\rho}_{ij}$ of columns i and j of the active dataset, clipped to $[-1, 1]$.
- `partial_correlation(i, j, Z)` regresses X_i and X_j on $[1, X_Z]$ via OLS (`numpy.linalg.lstsq`) and returns the Pearson correlation of the residuals; with $Z = \emptyset$ it reduces to `correlation(i, j)`.
- `independence_test(i, j, Z,  $\alpha$ )` performs the Fisher- z test:

$$\hat{\rho} = \text{partial_correlation}(i, j, Z), \quad z = \frac{1}{2} \log \frac{1 + \hat{\rho}}{1 - \hat{\rho}} \sqrt{n - |Z| - 3}, \quad p = \text{erfc}(|z|/\sqrt{2}), \quad (17)$$

where $\hat{\rho}$ is first clipped to $\pm(1 - 10^{-6})$ for numerical stability. The tool returns the pair $(p > \alpha, p)$, declaring conditional independence when $p > \alpha$. The test errors out when $\text{dof} = n - |Z| - 3 \leq 0$.

D.3 System prompts

Two system prompts are used. The *raw* prompt (`build_system_prompt_raw`) tells the model that it is performing causal discovery over an unknown linear-Gaussian DAG with full observability and no hidden confounders; cautions that dependence and correlation are not direct causation by themselves, so chains and shared causes can induce spurious dependence; defines an intervention as forcing one variable to a fixed value, breaking its incoming dependencies while leaving other relationships intact; suggests choosing intervention values far enough from the observational mean to make downstream shifts detectable while staying on the observed scale; lists the allowed actions; and instructs the model to use the action tool exactly once per turn and not to request hidden metadata.

The *stats* prompt (`build_system_prompt_stats`) uses the same base and adds three blocks. A *tool purpose* block: `correlation` is a coarse association check that is not causal direction by

itself, `partial_correlation` tests direct dependence after conditioning, and `independence_test` decides whether evidence supports conditional independence. A *tool data source* block: tools use the observational dataset before any intervention and the latest intervention dataset after one, and may return `value=null` with an error when a statistic is undefined. A *decision policy* block: gather evidence, orient edges when the evidence supports direction, and leave undirected only when evidence is ambiguous; in observational-only configurations the model is also told to perform at least three statistical actions before its first `submit_graph`.

The session prompt (`build_session_prompt`) is a JSON object with keys `variables`, `budget`, `output_schema`, `json_contract`, optionally `observational_data`, and any additional `session_context` entries; the contract requires every output-schema field to appear on every turn, with `null` for unused scalar fields, `[]` for unused list fields, and a brief `reasoning_summary`.

D.4 Observation format

The model’s input is *numerical*: the observational sample is delivered as a $n_{\text{obs}} \times d$ JSON array of raw floating-point values, not as natural-language descriptions, summary statistics, or QA pairs. Variables are referred to by index-encoded names $X_0, X_1, \dots, X_{(d-1)}$ carrying no domain semantics; there is no metadata that hints at the underlying graph. Every turn rebuilds the session-prompt JSON object from scratch with the following keys:

- `variables`: list of d string names `["X0", "X1", ...]`.
- `budget`: integer count of remaining intervention turns (decremented after each `intervene`).
- `output_schema / json_contract`: per-action field requirements as in §D.3.
- `observational_data` (when `include_observational_data=True`): $n_{\text{obs}} \times d$ list of lists of float, full machine precision. Withheld for `pc_cpdag_llm`, where the model receives only the PC partial graph and an `observational_summary` of per-variable means and standard deviations to three decimals (§D.6).
- `intervention_history` (after the first `intervene`): list of `{step, var, value, data, remaining_budget}` entries, where `data` is the $n_{\text{int}} \times d$ raw float matrix from that intervention.
- `working_memory`: the deque of prior `reasoning_summary` strings (max 12, ≤ 300 chars each; §D.7).

For the canonical L3 instance of Appendix F.2 ($d = 10, n_{\text{obs}} = 50, n_{\text{int}} = 25$), the step-1 user message is roughly 5 kB of JSON. After one intervention the step-2 message expands to roughly 15 kB as the 25×10 intervention sample is appended to `intervention_history`; subsequent interventions concatenate further blocks. There is no token-budget pruning: the model sees the entire numerical history every turn. The first ≤ 250 characters of the step-1 `observational_data` value are reproduced verbatim below to make the format unambiguous (full 50×10 payload available in the on-disk `events.jsonl` per Appendix G.5):

```
"observational_data": [[-0.5996848527,0.1257563716,0.3868730550, ... ,1.1959564656],
                      [ 0.2278882859,0.2679279303,2.1911019666, ... ,-0.2995619661],
                      [-0.2708565131,1.9276286132,-0.7202341092, ... , 0.1743317379],
                      ... // 47 more rows
                      ]
```

This design makes the benchmark a test of *structure recovery from raw numerical evidence*: the model must perform the regression / mean-shift / partial-correlation reasoning itself (or invoke the statistical primitives of §D.2 when allowed), not pattern-match against a pre-summarised description. Appendix F shows full agent traces operating on this format.

The raw-numerical format is a deliberate design choice, not a convenience. Pre-summarised inputs — correlation tables, natural-language descriptions of dependencies, or rounded statistics — would conflate causal reasoning with reading comprehension, and would make the `llm_raw`, `llm_stats`, and `pc_cpdag_llm` contrasts of §D.6 meaningless: every policy would already start from a summary, so the score gap between them would no longer measure what the model extracts from raw evidence. By delivering raw samples, the benchmark forces the model to perform the inference itself when allowed only `intervene/submit_graph`, and to choose *which* statistical queries to issue when

the stats tools are available; the resulting policy gaps reported in Appendix E measure exactly that increment of value. The cost of including the full numerical history every turn (§E.5) is reported alongside the score so a reader can judge the dollar-per-F1-point tradeoff this design imposes.

D.5 z -calibrated mean-shift rule

The classical resolver (`resolve_with_interventions`, `run_ladder.py:278-314`) takes a partially directed graph with directed edge set E_d and undirected edge set E_u , the observational sample D^{obs} , and an environment with a remaining intervention budget. It iterates the following three steps while E_u is non-empty and the budget is positive.

Target selection. Let $\deg_U(j)$ be the degree of node j in the current undirected subgraph. Pick

$$j^* = \arg \min_{j \in V_U} (-\deg_U(j), j), \quad (18)$$

i.e. the node of highest undirected degree, with ties broken by lowest node index (`run_ladder.py:289-293`).

Intervention. Set $v = \hat{\mu}_{\text{obs}}[j^*] + \delta$ with $\delta = 3.0$ (`INTERVENTION_OFFSET`, `run_ladder.py:56`) and call `env.intervene( $j^*$ ,  $v$ )` to draw n_{int} samples.

Orientation. For each undirected edge $\{j^*, k\} \in E_u$, compute the threshold

$$\tau_k = z_{0.975} \sqrt{\frac{\hat{\sigma}_{\text{obs}}^2[k]}{n_{\text{obs}}} + \frac{\hat{\sigma}_{\text{int}}^2[k]}{n_{\text{int}}}}, \quad z_{0.975} = 1.959963984540054, \quad (19)$$

where the variances are sample variances with `ddof = 1` (`run_ladder.py:285, 298`) and $z_{0.975}$ is `MEAN_SHIFT_Z_975` (`run_ladder.py:55`). Orient $j^* \rightarrow k$ if $|\hat{\mu}_{\text{int}}[k] - \hat{\mu}_{\text{obs}}[k]| > \tau_k$, otherwise $k \rightarrow j^*$. If the chosen orientation would create a directed cycle in E_d , the edge is left undirected and the loop continues (`run_ladder.py:310-311`, using `would_create_cycle`).

D.6 Per-policy specification

The four LLM policies share the loop in Appendix D.7; they differ only in the allowed-action set (`allowed_actions_for_method`, `tool_schema.py:102-128`), the system prompt, the session-prompt context, and the action budget (`-max-steps-raw`, default 20; `-max-steps-stats`, default 40; `run_ladder.py:992-993`).

pc_greedy (`run_ladder.py:436-453`). No LLM. The PC algorithm with Fisher- z conditional-independence tests at $\alpha = 0.05$ runs on D^{obs} via `causallearn`, producing an initial directed/undirected partition; the resolver of Appendix D.5 then consumes the full intervention budget.

llm_raw. Allowed actions `{intervene, submit_graph}`; raw prompt; the raw observational matrix is included in the session prompt; budget `-max-steps-raw`.

llm_stats. Allowed actions `{correlation, partial_correlation, independence_test, intervene, submit_graph}`; stats prompt; same observational data as `llm_raw`; budget `-max-steps-stats`.

pc_cpdag_llm (`run_ladder.py:727-772`). PC runs first on D^{obs} . The LLM session is then initialised with `include_observational_data=False` — the raw matrix is withheld — and a `session_context` containing the PC `initial_directed_edges` and `initial_undirected_edges`, an `observational_summary` with per-variable means and standard deviations rounded to three decimals, and an instruction to start from the supplied PC partial graph and use interventions to resolve uncertain directions. Allowed actions `{intervene, submit_graph}`; raw prompt; budget `-max-steps-raw`.

1009 `llm_stats_cpdag_greedy` (`run_ladder.py:775-811`). Allowed actions `{correlation,`
1010 `partial_correlation, independence_test, submit_graph}` — `intervene` is removed; stats
1011 prompt; `budget -max-steps-stats`. After the LLM submits, the resolver of Appendix D.5 runs post-
1012 hoc on the LLM’s E_d and E_u against the remaining intervention budget; the additional intervention
1013 count is added to `intervene_actions` in the run diagnostics.

1014 D.7 Loop semantics

1015 LLM episodes execute as a stateful sequential loop. The session prompt is rebuilt every turn
1016 from the current `remaining_budget` and the (possibly coerced) effective allowed-action set;
1017 tool history accumulates across turns; the working-memory deque keeps at most 12 reasoning
1018 summaries of ≤ 300 characters each (`llm.py:28-29`). On the final step the runtime forces
1019 `effective_allowed_actions = {submit_graph}` regardless of the policy’s nominal action set,
1020 so the episode always terminates with a submission (`run_ladder.py:531-532`). At submission
1021 time, undirected edges that overlap a submitted directed edge are silently dropped before scoring; the
1022 dropped count is logged as `sanitized_overlap_count` (`run_ladder.py:259-275, 664-668`).
1023 This enforces the disjointness invariant of Appendix B even when the model violates it.

1024 E Full results: per-model, per-level, per-policy

1025 This appendix is the full numerical record behind Section 5. Every cell is aggregated by the protocol
1026 of §E.1 from the canonical `results_long.csv` traces of Appendix G.2; the random-floor reference
1027 column uses the closed form of Appendix C.2; the score axes are defined in Appendix B and the
1028 policies in Appendix D. Five models, six levels (L0–L5), and the four active LLM policies plus the
1029 `pc_greedy` baseline are reported, with the per-cell sample count n included so a reader can read the
1030 standard errors honestly. §5 reports the cross-model average and one layered slice; everything else —
1031 per-level breakdowns, score-axis disagreements, gap-above-floor, and dollar costs — is here.

1032 E.1 Aggregation protocol

1033 For each panel directory of Appendix G.2, every per-instance row of `results_long.csv` is filtered to
1034 `status == "success"` and `panel == "active"`, then de-duplicated on the key (`level, seed,`
1035 `panel, method, model`) keeping the row with the latest `timestamp_utc`. This matches the
1036 de-duplication used by `scripts/make_paper_figures.py:load_long`. Within each (`panel,`
1037 `level, method`) cell, the mean and standard error of every metric are reported as

$$\bar{x} = \frac{1}{n} \sum_{i=1}^n x_i, \quad \text{SE}(\bar{x}) = \frac{\hat{\sigma}_{n-1}}{\sqrt{n}}, \quad (20)$$

1038 where n is the number of successful, deduplicated rows for that cell. The gap-above-floor col-
1039 umn is $\overline{F1_{\text{dir}}} - \mathbb{E}[F_1 \mid d, k]$ with the closed-form floor of Appendix C.2 evaluated at the pro-
1040 duction (d, k) of the level. The compelled-edge column is the canonical `compelled_f1` from
1041 `src/causal_discovery/scoring/scores.py` aggregated identically; it is present per row in
1042 `results_long.csv` but absent from the harness-emitted `results_summary.csv`, so the per-panel
1043 summary CSVs are not used here. The aggregation script is `scripts/aggregate_panels.py`;
1044 its output is `traces/aggregated/per_model_per_level_per_method.csv` and is the single
1045 source for every number in §E.2 through §E.5.

1046 **Cross-panel `pc_greedy` baseline.** The classical `pc_greedy` policy does not depend on the LLM,
1047 so its per-level numbers are identical across all five panels (same seed map, deterministic algorithm;
1048 verified bit-exact in the aggregated CSV). To avoid restating the same five rows in every model
1049 subsection, `pc_greedy` is reported *once* in the cross-model pivot of §E.3 and not duplicated in
1050 the per-model tables of §E.2. The per-model prose still compares LLM policies to `pc_greedy` by
1051 referring to those shared values.

1052 E.2 Per-model layered tables

1053 Each subsection below contains one table with rows = (level, method) and columns = n , skeleton F_1 ,
1054 compelled-edge F_1 , directed F_1 , gap above floor, SHD. Mean $\pm$ SE; three decimals; n may be less
1055 than 8 at higher levels when the auto-resume loop (Appendix G.1) failed to recover all seeds within
1056 the retry budget.

1057 **E.2.1 GPT-5.5**

Lvl	Method	n	skel F_1	comp F_1	dir F_1	gap	SHD
L0	llm_raw	8	1.000 $\pm$ 0.000	1.000 $\pm$ 0.000	1.000 $\pm$ 0.000	+0.785	0.00 $\pm$ 0.00
	llm_stats	8	1.000 $\pm$ 0.000	1.000 $\pm$ 0.000	1.000 $\pm$ 0.000	+0.785	0.00 $\pm$ 0.00
	pc_cpdag_llm	8	0.975 $\pm$ 0.025	1.000 $\pm$ 0.000	0.892 $\pm$ 0.084	+0.676	0.38 $\pm$ 0.26
	llm_stats_cpdag_greedy	8	0.854 $\pm$ 0.073	1.000 $\pm$ 0.000	0.792 $\pm$ 0.125	+0.576	0.88 $\pm$ 0.44
L1	llm_raw	8	0.881 $\pm$ 0.043	0.907 $\pm$ 0.036	0.824 $\pm$ 0.078	+0.629	1.75 $\pm$ 0.73
	llm_stats	8	0.848 $\pm$ 0.049	0.852 $\pm$ 0.059	0.783 $\pm$ 0.095	+0.588	2.12 $\pm$ 0.81
	pc_cpdag_llm	8	0.873 $\pm$ 0.025	0.810 $\pm$ 0.059	0.827 $\pm$ 0.047	+0.632	1.62 $\pm$ 0.38
	llm_stats_cpdag_greedy	8	0.609 $\pm$ 0.079	0.613 $\pm$ 0.114	0.528 $\pm$ 0.104	+0.332	4.00 $\pm$ 0.57
L2	llm_raw	8	0.792 $\pm$ 0.040	0.743 $\pm$ 0.069	0.734 $\pm$ 0.050	+0.560	4.25 $\pm$ 0.82
	llm_stats	8	0.782 $\pm$ 0.050	0.725 $\pm$ 0.079	0.704 $\pm$ 0.070	+0.530	4.12 $\pm$ 0.93
	pc_cpdag_llm	8	0.760 $\pm$ 0.040	0.528 $\pm$ 0.117	0.666 $\pm$ 0.064	+0.493	4.38 $\pm$ 0.71
	llm_stats_cpdag_greedy	8	0.529 $\pm$ 0.091	0.536 $\pm$ 0.104	0.425 $\pm$ 0.075	+0.252	8.00 $\pm$ 0.96
L3	llm_raw	8	0.772 $\pm$ 0.027	0.753 $\pm$ 0.039	0.705 $\pm$ 0.036	+0.551	6.00 $\pm$ 0.73
	llm_stats	8	0.661 $\pm$ 0.053	0.593 $\pm$ 0.084	0.628 $\pm$ 0.061	+0.474	7.75 $\pm$ 1.31
	pc_cpdag_llm	8	0.779 $\pm$ 0.031	0.626 $\pm$ 0.055	0.688 $\pm$ 0.047	+0.533	5.50 $\pm$ 0.63
	llm_stats_cpdag_greedy	8	0.509 $\pm$ 0.101	0.433 $\pm$ 0.079	0.423 $\pm$ 0.085	+0.268	8.88 $\pm$ 0.91
L4	llm_raw	8	0.713 $\pm$ 0.031	0.673 $\pm$ 0.052	0.686 $\pm$ 0.025	+0.553	7.12 $\pm$ 0.55
	llm_stats	8	0.646 $\pm$ 0.044	0.645 $\pm$ 0.039	0.578 $\pm$ 0.050	+0.445	9.62 $\pm$ 1.39
	pc_cpdag_llm	8	0.775 $\pm$ 0.024	0.557 $\pm$ 0.078	0.679 $\pm$ 0.051	+0.546	6.38 $\pm$ 0.80
	llm_stats_cpdag_greedy	8	0.386 $\pm$ 0.087	0.347 $\pm$ 0.093	0.306 $\pm$ 0.059	+0.173	12.88 $\pm$ 0.85
L5	llm_raw	8	0.606 $\pm$ 0.036	0.582 $\pm$ 0.055	0.537 $\pm$ 0.055	+0.420	12.25 $\pm$ 1.19
	llm_stats	8	0.586 $\pm$ 0.030	0.552 $\pm$ 0.067	0.508 $\pm$ 0.053	+0.391	12.75 $\pm$ 1.33
	pc_cpdag_llm	8	0.759 $\pm$ 0.026	0.553 $\pm$ 0.073	0.646 $\pm$ 0.052	+0.529	8.12 $\pm$ 0.85
	llm_stats_cpdag_greedy	8	0.464 $\pm$ 0.058	0.345 $\pm$ 0.077	0.372 $\pm$ 0.048	+0.255	17.00 $\pm$ 1.45

1059 GPT-5.5 is the only panel where end-to-end llm_raw keeps pace with pc_greedy on directed F_1
1060 across the full ladder, and where the CPDAG-prior policy pc_cpdag_llm adds essentially nothing
1061 (gap delta below 0.01 at every level except L0). The compelled-edge F_1 tells a slightly different story:
1062 llm_raw beats pc_greedy on comp_f1 at every level L1–L4 by 0.10–0.22, indicating the model
1063 orients the observationally identifiable edges more reliably than the classical pipeline even when their
1064 directed- F_1 scores are close. The llm_stats_cpdag_greedy policy is the only one that visibly
1065 degrades — removing the intervention budget from the LLM and handing it back to the post-hoc
1066 resolver drops the score by ≈ 0.3 at every level.

1067 **E.2.2 GPT-5.4-mini**

Lvl	Method	n	skel F_1	comp F_1	dir F_1	gap	SHD
L0	llm_raw	9	0.475 $\pm$ 0.062	1.000 $\pm$ 0.000	0.227 $\pm$ 0.045	+0.012	4.33 $\pm$ 0.17
	llm_stats	8	0.625 $\pm$ 0.082	1.000 $\pm$ 0.000	0.338 $\pm$ 0.113	+0.123	3.62 $\pm$ 0.56
	pc_cpdag_llm	8	0.957 $\pm$ 0.029	1.000 $\pm$ 0.000	0.411 $\pm$ 0.105	+0.195	1.88 $\pm$ 0.35
	llm_stats_cpdag_greedy	8	0.531 $\pm$ 0.054	1.000 $\pm$ 0.000	0.490 $\pm$ 0.084	+0.274	2.75 $\pm$ 0.41
L1	llm_raw	8	0.480 $\pm$ 0.048	0.342 $\pm$ 0.122	0.240 $\pm$ 0.066	+0.044	7.62 $\pm$ 0.60
	llm_stats	8	0.404 $\pm$ 0.069	0.483 $\pm$ 0.125	0.252 $\pm$ 0.081	+0.057	11.38 $\pm$ 0.86
	pc_cpdag_llm	8	0.817 $\pm$ 0.040	0.632 $\pm$ 0.124	0.410 $\pm$ 0.086	+0.214	4.38 $\pm$ 0.60
	llm_stats_cpdag_greedy	8	0.138 $\pm$ 0.052	0.175 $\pm$ 0.128	0.138 $\pm$ 0.052	−0.057	6.00 $\pm$ 0.33
L2	llm_raw	8	0.266 $\pm$ 0.054	0.174 $\pm$ 0.114	0.090 $\pm$ 0.052	−0.084	14.88 $\pm$ 0.77
	llm_stats	8	0.286 $\pm$ 0.067	0.108 $\pm$ 0.084	0.083 $\pm$ 0.045	−0.091	15.75 $\pm$ 2.46
	pc_cpdag_llm	8	0.740 $\pm$ 0.037	0.406 $\pm$ 0.146	0.414 $\pm$ 0.096	+0.241	6.38 $\pm$ 0.89
	llm_stats_cpdag_greedy	8	0.025 $\pm$ 0.025	0.050 $\pm$ 0.050	0.025 $\pm$ 0.025	−0.148	9.50 $\pm$ 0.27
L3	llm_raw	8	0.330 $\pm$ 0.044	0.271 $\pm$ 0.070	0.167 $\pm$ 0.041	+0.012	24.62 $\pm$ 3.24
	llm_stats	8	0.278 $\pm$ 0.046	0.121 $\pm$ 0.064	0.082 $\pm$ 0.046	−0.073	19.00 $\pm$ 1.27
	pc_cpdag_llm	8	0.763 $\pm$ 0.036	0.489 $\pm$ 0.075	0.465 $\pm$ 0.077	+0.311	8.12 $\pm$ 0.97
	llm_stats_cpdag_greedy	8	0.038 $\pm$ 0.025	0.028 $\pm$ 0.028	0.038 $\pm$ 0.025	−0.116	12.00 $\pm$ 0.27
L4	llm_raw	8	0.265 $\pm$ 0.032	0.279 $\pm$ 0.061	0.156 $\pm$ 0.030	+0.023	27.12 $\pm$ 1.97
	llm_stats	8	0.155 $\pm$ 0.052	0.087 $\pm$ 0.049	0.076 $\pm$ 0.036	−0.057	25.75 $\pm$ 6.27
	pc_cpdag_llm	8	0.764 $\pm$ 0.027	0.518 $\pm$ 0.064	0.505 $\pm$ 0.034	+0.372	9.00 $\pm$ 0.63
	llm_stats_cpdag_greedy	8	0.033 $\pm$ 0.022	0.000 $\pm$ 0.000	0.033 $\pm$ 0.022	−0.100	14.38 $\pm$ 0.32
L5	llm_raw	8	0.236 $\pm$ 0.021	0.146 $\pm$ 0.047	0.070 $\pm$ 0.023	−0.046	37.50 $\pm$ 5.10
	llm_stats	8	0.099 $\pm$ 0.023	0.039 $\pm$ 0.026	0.021 $\pm$ 0.014	−0.095	21.38 $\pm$ 1.76
	pc_cpdag_llm	8	0.746 $\pm$ 0.022	0.364 $\pm$ 0.105	0.349 $\pm$ 0.084	+0.232	12.88 $\pm$ 1.17
	llm_stats_cpdag_greedy	8	0.028 $\pm$ 0.018	0.058 $\pm$ 0.039	0.028 $\pm$ 0.018	−0.089	16.50 $\pm$ 0.27

1069 GPT-5.4-mini is the panel where the structural-prior cut matters most: pc_cpdag_llm clears the
1070 floor at every level by +0.21 to +0.37, while llm_raw sits at or below the floor from L2 onward.

1071 llm_stats_cpdag_greedy is uniformly below the floor from L1 to L5 — removing intervention
1072 from a small model on top of the stats-only stack collapses to near-zero submissions ($n = 8$ runs
1073 producing ≤ 1 correct directed edge on average). The model is informative only with a CPDAG
1074 handed to it; it is not a competent end-to-end discoverer at this scale.

1075 E.2.3 Sonnet-4.6

Lvl	Method	n	skel F_1	comp F_1	dir F_1	gap	SHD
L0	llm_raw	8	0.714 ± 0.070	1.000 ± 0.000	0.554 ± 0.070	$+0.338$	2.25 ± 0.37
	llm_stats	8	0.690 ± 0.067	1.000 ± 0.000	0.363 ± 0.094	$+0.148$	2.88 ± 0.40
	pc_cpdag_llm	8	0.975 ± 0.025	1.000 ± 0.000	0.808 ± 0.086	$+0.593$	0.62 ± 0.26
	llm_stats_cpdag_greedy	6	0.921 ± 0.056	1.000 ± 0.000	0.762 ± 0.115	$+0.547$	1.00 ± 0.45
L1	llm_raw	8	0.514 ± 0.064	0.558 ± 0.107	0.426 ± 0.094	$+0.230$	6.00 ± 0.87
	llm_stats	8	0.498 ± 0.067	0.321 ± 0.134	0.164 ± 0.060	-0.031	9.12 ± 0.93
	pc_cpdag_llm	8	0.825 ± 0.040	0.786 ± 0.061	0.742 ± 0.040	$+0.546$	2.38 ± 0.38
	llm_stats_cpdag_greedy	5	0.744 ± 0.052	0.607 ± 0.127	0.418 ± 0.134	$+0.223$	5.20 ± 1.24
L2	llm_raw	7	0.433 ± 0.053	0.311 ± 0.100	0.232 ± 0.075	$+0.059$	11.00 ± 0.98
	llm_stats	7	0.398 ± 0.048	0.321 ± 0.099	0.187 ± 0.058	$+0.014$	12.71 ± 0.78
	pc_cpdag_llm	8	0.740 ± 0.037	0.528 ± 0.117	0.585 ± 0.055	$+0.411$	5.12 ± 0.61
	llm_stats_cpdag_greedy	4	0.547 ± 0.054	0.392 ± 0.171	0.374 ± 0.133	$+0.200$	8.75 ± 1.03
L3	llm_raw	6	0.446 ± 0.024	0.377 ± 0.070	0.319 ± 0.063	$+0.164$	13.83 ± 1.35
	llm_stats	8	0.316 ± 0.059	0.269 ± 0.079	0.233 ± 0.066	$+0.079$	14.38 ± 0.78
	pc_cpdag_llm	8	0.779 ± 0.031	0.563 ± 0.067	0.629 ± 0.046	$+0.474$	6.38 ± 0.60
	llm_stats_cpdag_greedy	4	0.124 ± 0.083	0.129 ± 0.081	0.095 ± 0.058	-0.060	12.50 ± 0.29
L4	llm_raw	5	0.363 ± 0.069	0.273 ± 0.089	0.294 ± 0.048	$+0.161$	15.20 ± 0.97
	llm_stats	8	0.311 ± 0.049	0.272 ± 0.070	0.206 ± 0.036	$+0.073$	18.62 ± 0.94
	pc_cpdag_llm	8	0.786 ± 0.029	0.594 ± 0.074	0.606 ± 0.050	$+0.473$	7.50 ± 0.78
	llm_stats_cpdag_greedy	6	0.212 ± 0.081	0.195 ± 0.102	0.217 ± 0.084	$+0.084$	14.33 ± 1.23
L5	llm_raw	4	0.315 ± 0.060	0.358 ± 0.076	0.252 ± 0.100	$+0.135$	21.25 ± 3.04
	llm_stats	8	0.235 ± 0.043	0.188 ± 0.068	0.127 ± 0.049	$+0.010$	22.00 ± 1.72
	pc_cpdag_llm	8	0.756 ± 0.022	0.538 ± 0.087	0.569 ± 0.071	$+0.452$	9.25 ± 1.10
	llm_stats_cpdag_greedy	5	0.297 ± 0.034	0.251 ± 0.066	0.237 ± 0.037	$+0.120$	18.00 ± 1.84

1077 Sonnet-4.6 falls between GPT-5.5 and the smaller models: pc_cpdag_llm stays within 0.07–0.16 of
1078 pc_greedy on directed F_1 at every level, and llm_raw clears the floor at all six levels but trails the
1079 CPDAG-prior policy by 0.30–0.40. llm_stats at L1 sits below the random floor (-0.031), the only
1080 sub-floor cell in this panel and a sign that exposing the model to coarse association tools without an
1081 intervention budget actively misleads it on the smallest active level. The lower n values at L2–L5
1082 (4–7 successful seeds per cell) come from OpenRouter rate-limit failures during the run, retried but
1083 not always recovered.

1084 E.2.4 Haiku-4.5

Lvl	Method	n	skel F_1	comp F_1	dir F_1	gap	SHD
L0	llm_raw	8	0.576 $\pm$ 0.102	1.000 $\pm$ 0.000	0.309 $\pm$ 0.120	+0.094	3.50 $\pm$ 0.63
	llm_stats	8	0.637 $\pm$ 0.055	1.000 $\pm$ 0.000	0.442 $\pm$ 0.039	+0.226	3.12 $\pm$ 0.48
	pc_cpdag_llm	8	0.975 $\pm$ 0.025	1.000 $\pm$ 0.000	0.400 $\pm$ 0.116	+0.185	1.88 $\pm$ 0.35
	llm_stats_cpdag_greedy	8	0.826 $\pm$ 0.043	1.000 $\pm$ 0.000	0.433 $\pm$ 0.113	+0.218	2.62 $\pm$ 0.50
L1	llm_raw	7	0.401 $\pm$ 0.078	0.333 $\pm$ 0.141	0.192 $\pm$ 0.071	-0.003	9.00 $\pm$ 0.98
	llm_stats	8	0.523 $\pm$ 0.066	0.342 $\pm$ 0.141	0.219 $\pm$ 0.073	+0.023	6.88 $\pm$ 0.90
	pc_cpdag_llm	8	0.811 $\pm$ 0.042	0.761 $\pm$ 0.053	0.599 $\pm$ 0.050	+0.403	3.25 $\pm$ 0.37
	llm_stats_cpdag_greedy	8	0.679 $\pm$ 0.062	0.613 $\pm$ 0.114	0.375 $\pm$ 0.057	+0.180	5.88 $\pm$ 0.83
L2	llm_raw	6	0.273 $\pm$ 0.079	0.227 $\pm$ 0.115	0.160 $\pm$ 0.058	-0.014	13.50 $\pm$ 0.89
	llm_stats	8	0.296 $\pm$ 0.080	0.249 $\pm$ 0.100	0.183 $\pm$ 0.061	+0.010	10.62 $\pm$ 1.35
	pc_cpdag_llm	7	0.717 $\pm$ 0.053	0.401 $\pm$ 0.168	0.369 $\pm$ 0.101	+0.196	6.57 $\pm$ 0.90
	llm_stats_cpdag_greedy	5	0.554 $\pm$ 0.086	0.503 $\pm$ 0.141	0.377 $\pm$ 0.057	+0.204	8.40 $\pm$ 0.98
L3	llm_raw	8	0.292 $\pm$ 0.028	0.095 $\pm$ 0.036	0.085 $\pm$ 0.032	-0.069	19.38 $\pm$ 1.21
	llm_stats	8	0.168 $\pm$ 0.066	0.063 $\pm$ 0.031	0.051 $\pm$ 0.026	-0.104	16.88 $\pm$ 3.90
	pc_cpdag_llm	8	0.779 $\pm$ 0.031	0.481 $\pm$ 0.081	0.502 $\pm$ 0.057	+0.347	7.62 $\pm$ 0.68
	llm_stats_cpdag_greedy	7	0.502 $\pm$ 0.036	0.339 $\pm$ 0.080	0.321 $\pm$ 0.047	+0.166	14.43 $\pm$ 1.65
L4	llm_raw	8	0.217 $\pm$ 0.034	0.134 $\pm$ 0.061	0.103 $\pm$ 0.030	-0.030	22.50 $\pm$ 0.98
	llm_stats	8	0.156 $\pm$ 0.033	0.044 $\pm$ 0.029	0.075 $\pm$ 0.032	-0.058	19.50 $\pm$ 5.11
	pc_cpdag_llm	8	0.763 $\pm$ 0.019	0.527 $\pm$ 0.065	0.447 $\pm$ 0.065	+0.314	9.62 $\pm$ 0.86
	llm_stats_cpdag_greedy	6	0.450 $\pm$ 0.074	0.275 $\pm$ 0.089	0.258 $\pm$ 0.050	+0.125	19.83 $\pm$ 3.39
L5	llm_raw	8	0.230 $\pm$ 0.043	0.081 $\pm$ 0.036	0.047 $\pm$ 0.020	-0.069	27.38 $\pm$ 3.21
	llm_stats	8	0.131 $\pm$ 0.037	0.031 $\pm$ 0.021	0.026 $\pm$ 0.017	-0.091	17.25 $\pm$ 0.45
	pc_cpdag_llm	8	0.753 $\pm$ 0.026	0.481 $\pm$ 0.091	0.465 $\pm$ 0.074	+0.348	10.75 $\pm$ 1.13
	llm_stats_cpdag_greedy	7	0.222 $\pm$ 0.034	0.159 $\pm$ 0.049	0.113 $\pm$ 0.036	-0.004	19.43 $\pm$ 1.09

1086 Haiku-4.5 fails to clear the floor under both end-to-end LLM policies from L3 onward, and its
 1087 llm_raw is below the floor at L1, L2, L4, L5; only pc_cpdag_llm carries the panel to a meaningful
 1088 gap. The model is essentially submitting structured noise once $d \geq 10$, but with a CPDAG it
 1089 still recovers 0.45–0.60 directed F_1 , indicating the orientation step alone is within reach. The
 1090 llm_stats_cpdag_greedy hybrid recovers some of that ground at L1–L4 but collapses again at
 1091 L5 (−0.004 gap).

1092 E.2.5 Gemini-3-Flash

Lvl	Method	n	skel F_1	comp F_1	dir F_1	gap	SHD
L0	llm_raw	8	0.729 $\pm$ 0.037	1.000 $\pm$ 0.000	0.519 $\pm$ 0.110	+0.304	2.62 $\pm$ 0.46
	llm_stats	8	0.714 $\pm$ 0.070	1.000 $\pm$ 0.000	0.393 $\pm$ 0.143	+0.178	2.88 $\pm$ 0.58
	pc_cpdag_llm	8	0.875 $\pm$ 0.125	1.000 $\pm$ 0.000	0.625 $\pm$ 0.133	+0.410	1.12 $\pm$ 0.40
	llm_stats_cpdag_greedy	8	0.946 $\pm$ 0.054	1.000 $\pm$ 0.000	0.792 $\pm$ 0.140	+0.576	0.88 $\pm$ 0.64
L1	llm_raw	7	0.603 $\pm$ 0.052	0.669 $\pm$ 0.100	0.414 $\pm$ 0.074	+0.219	6.14 $\pm$ 0.91
	llm_stats	8	0.645 $\pm$ 0.042	0.617 $\pm$ 0.123	0.345 $\pm$ 0.057	+0.149	6.12 $\pm$ 0.55
	pc_cpdag_llm	8	0.848 $\pm$ 0.042	0.810 $\pm$ 0.059	0.709 $\pm$ 0.060	+0.513	2.38 $\pm$ 0.46
	llm_stats_cpdag_greedy	8	0.728 $\pm$ 0.024	0.683 $\pm$ 0.078	0.579 $\pm$ 0.085	+0.383	4.25 $\pm$ 0.56
L2	llm_raw	7	0.466 $\pm$ 0.051	0.413 $\pm$ 0.138	0.292 $\pm$ 0.071	+0.119	12.71 $\pm$ 1.04
	llm_stats	8	0.439 $\pm$ 0.068	0.393 $\pm$ 0.104	0.254 $\pm$ 0.076	+0.081	12.12 $\pm$ 1.46
	pc_cpdag_llm	8	0.740 $\pm$ 0.037	0.528 $\pm$ 0.117	0.495 $\pm$ 0.085	+0.321	5.75 $\pm$ 0.80
	llm_stats_cpdag_greedy	8	0.553 $\pm$ 0.069	0.585 $\pm$ 0.082	0.511 $\pm$ 0.073	+0.337	7.88 $\pm$ 1.12
L3	llm_raw	7	0.418 $\pm$ 0.036	0.414 $\pm$ 0.087	0.245 $\pm$ 0.040	+0.091	21.71 $\pm$ 2.20
	llm_stats	5	0.404 $\pm$ 0.041	0.465 $\pm$ 0.129	0.301 $\pm$ 0.079	+0.146	20.20 $\pm$ 2.91
	pc_cpdag_llm	8	0.779 $\pm$ 0.031	0.510 $\pm$ 0.081	0.540 $\pm$ 0.057	+0.385	7.00 $\pm$ 0.71
	llm_stats_cpdag_greedy	8	0.493 $\pm$ 0.030	0.390 $\pm$ 0.050	0.460 $\pm$ 0.043	+0.306	11.25 $\pm$ 1.18
L4	llm_raw	6	0.372 $\pm$ 0.044	0.493 $\pm$ 0.063	0.281 $\pm$ 0.023	+0.148	27.00 $\pm$ 3.10
	llm_stats	7	0.320 $\pm$ 0.070	0.418 $\pm$ 0.102	0.226 $\pm$ 0.068	+0.093	22.43 $\pm$ 2.29
	pc_cpdag_llm	8	0.781 $\pm$ 0.027	0.587 $\pm$ 0.070	0.622 $\pm$ 0.050	+0.489	7.00 $\pm$ 0.80
	llm_stats_cpdag_greedy	8	0.433 $\pm$ 0.046	0.393 $\pm$ 0.077	0.403 $\pm$ 0.051	+0.270	12.38 $\pm$ 0.78
L5	llm_raw	8	0.333 $\pm$ 0.038	0.340 $\pm$ 0.069	0.185 $\pm$ 0.039	+0.069	28.62 $\pm$ 2.38
	llm_stats	8	0.254 $\pm$ 0.073	0.202 $\pm$ 0.089	0.158 $\pm$ 0.070	+0.042	24.00 $\pm$ 3.86
	pc_cpdag_llm	8	0.764 $\pm$ 0.029	0.513 $\pm$ 0.106	0.559 $\pm$ 0.063	+0.443	9.12 $\pm$ 1.03
	llm_stats_cpdag_greedy	8	0.338 $\pm$ 0.032	0.346 $\pm$ 0.066	0.315 $\pm$ 0.037	+0.199	15.62 $\pm$ 1.25

1094 Gemini-3-Flash clears the floor at every (level, method) cell, the only non-GPT-5.5 model with
 1095 that property, but its end-to-end llm_raw score still sits 0.30–0.45 below pc_cpdag_llm at every
 1096 level L1–L5. The llm_stats_cpdag_greedy hybrid is unusually strong here — the only panel
 1097 where stats-only LLM reasoning followed by the post-hoc resolver remains within 0.10–0.15 of the

LLM-with-CPDAG policy at L2–L5. SHD inflates faster than F_1 would suggest (e.g. L3 `llm_raw` SHD = 21.7 at $F_1 = 0.245$); the model submits many extra wrong edges rather than refusing to commit, an error mode the directed- F_1 ranking partially hides.

E.3 Per-level summary across models (`llm_raw`)

The cross-model summary in Section 5 averages over L1–L5; the per-level pivot below gives the dropoff with d that the average hides. Each cell is mean directed F_1 for the headline `llm_raw` policy, with the gap above the random floor of Appendix C.2 on the second line. The `pc_greedy` row is the shared baseline.

Level	GPT-5.5	GPT-5.4-mini	Sonnet-4.6	Haiku-4.5	Gemini-3-Flash
L1 <code>pc_greedy</code>			0.802/ + 0.607		
L2 <code>pc_greedy</code>			0.613/ + 0.439		
L3 <code>pc_greedy</code>			0.697/ + 0.542		
L4 <code>pc_greedy</code>			0.690/ + 0.557		
L5 <code>pc_greedy</code>			0.642/ + 0.525		
L1	0.824/ + 0.629	0.240/ + 0.044	0.426/ + 0.230	0.192/ − 0.003	0.414/ + 0.219
L2	0.734/ + 0.560	0.090/ − 0.084	0.232/ + 0.059	0.160/ − 0.014	0.292/ + 0.119
L3	0.705/ + 0.551	0.167/ + 0.012	0.319/ + 0.164	0.085/ − 0.069	0.245/ + 0.091
L4	0.686/ + 0.553	0.156/ + 0.023	0.294/ + 0.161	0.103/ − 0.030	0.281/ + 0.148
L5	0.537/ + 0.420	0.070/ − 0.046	0.252/ + 0.135	0.047/ − 0.069	0.185/ + 0.069

The per-level read changes the cross-model ranking. By level-mean directed F_1 averaged L1–L5, Sonnet-4.6 (0.305) leads Gemini-3-Flash (0.284); the body §5 reports the sample-weighted average over instances, which tracks the higher- n levels more closely (Sonnet 0.314, Gemini 0.281). By gap-above-floor at L5 specifically, Sonnet (+0.135) leads Gemini (+0.069) by a wider margin than either average suggests, because Gemini’s `llm_raw` drops faster relative to the floor as d grows. Two models cross the floor downward at some level: GPT-5.4-mini at L2 and L5, and Haiku-4.5 at L1, L2, L3, L4, L5.

E.4 Diagnostic figures

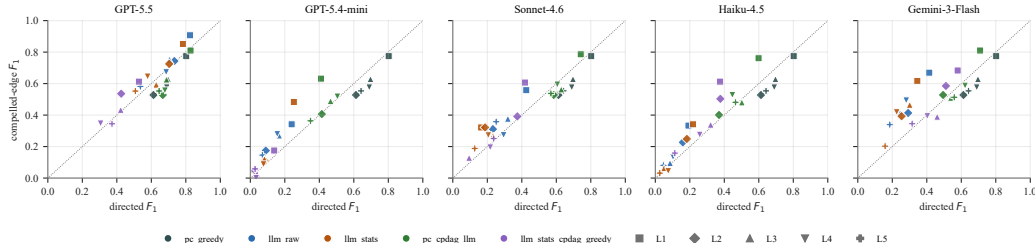


Figure 5: Compelled-edge F_1 versus directed F_1 per (level, method) cell, one sub-axis per model. Marker colour encodes method, marker shape encodes level. Diagonal: equal scores. Points above the diagonal correspond to submissions where the model orients the observationally identifiable (compelled) edges more reliably than the unrestricted directed set — they get the easy oriented edges right but submit many extra wrong directions. Most strongly visible for `llm_stats` on GPT-5.4-mini, Sonnet-4.6, and Haiku-4.5 (smaller models with the stats prompt produce orientation noise on top of correct compelled-edge inference).

The diagonal is the case where the two scores agree; cells well off the diagonal are where the score axes disagree on policy ranking, and that disagreement is the case for reporting both. `pc_greedy` sits below the diagonal in every panel (compelled < directed): the classical pipeline orients edges with the z -calibrated mean-shift rule applied uniformly, so directed F_1 benefits from correctly oriented non-compelled edges that compelled- F_1 does not credit. The opposite pattern — compelled > directed — appears mostly for `llm_stats` and `llm_raw` on the smaller models, where the compelled fraction of submissions is more reliably oriented than the rest.

This is the calibrated cross-model picture. The floor-subtracted view collapses the ladder dropoff that the raw F_1 tables make hard to read across rows of different density. Three shapes are visible: (i)

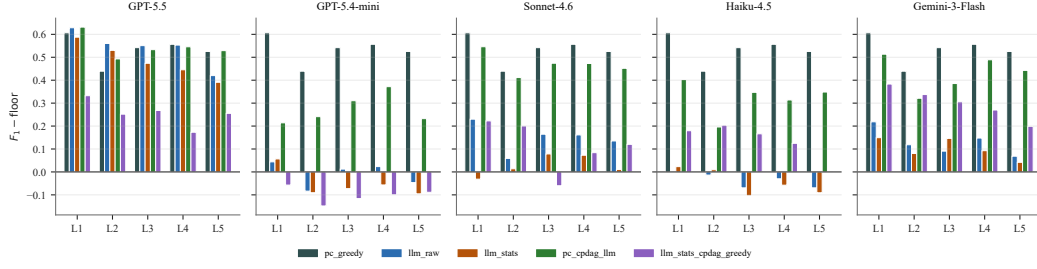


Figure 6: Gap above the random-submission directed- F_1 floor (Appendix C) per level per method, one sub-axis per model. Zero is the floor. Bars below zero are submissions worse than uniform random at that density.

GPT-5.5 keeps a near-flat ~ 0.5 gap across L1–L5 for the three competitive policies, (ii) Sonnet-4.6 and Gemini-3-Flash maintain positive gaps everywhere but lose ground on `llm_raw` as d grows, and (iii) GPT-5.4-mini and Haiku-4.5 produce sub-floor submissions on the end-to-end policies and only recover a positive gap with the CPDAG prior. The chart makes clear why the cross-model headline number averaged at L1–L5 is a poor summary: the spread within a model across levels is comparable to the spread across models at a single level.

E.5 Cost and action counts

The active LLM policies differ in token volume by 2–3 orders of magnitude across models, with the smaller models compensating for low directed- F_1 output via low cost per submission. The table reports averages over L1–L5 of `cost_usd_mean` and `total_actions_mean` per cell, plus the directed- F_1 -per-dollar ratio.

Model	Method	cost (\$)	actions	dir F_1	$F_1 / \$$
GPT-5.5	<code>llm_raw</code>	0.706	5.0	0.697	0.99
	<code>llm_stats</code>	0.927	10.4	0.640	0.69
	<code>pc_cpdag_llm</code>	0.178	3.8	0.701	3.95
	<code>llm_stats_cpdag_greedy</code>	0.161	5.8	0.411	2.56
GPT-5.4-mini	<code>llm_raw</code>	0.015	2.5	0.145	9.92
	<code>llm_stats</code>	0.015	4.3	0.103	6.89
	<code>pc_cpdag_llm</code>	0.005	2.3	0.429	80.5
	<code>llm_stats_cpdag_greedy</code>	0.009	4.8	0.053	5.93
Sonnet-4.6	<code>llm_raw</code>	0.191	2.9	0.305	1.60
	<code>llm_stats</code>	0.056	2.3	0.183	3.26
	<code>pc_cpdag_llm</code>	0.058	3.0	0.626	10.7
	<code>llm_stats_cpdag_greedy</code>	0.670	26.6	0.268	0.40
Haiku-4.5	<code>llm_raw</code>	0.028	3.0	0.118	4.20
	<code>llm_stats</code>	0.111	12.3	0.111	0.99
	<code>pc_cpdag_llm</code>	0.014	2.8	0.476	34.3
	<code>llm_stats_cpdag_greedy</code>	0.299	35.0	0.289	0.97
Gemini-3-Flash	<code>llm_raw</code>	0.025	3.1	0.284	11.5
	<code>llm_stats</code>	0.125	15.0	0.257	2.05
	<code>pc_cpdag_llm</code>	0.010	3.0	0.585	57.1
	<code>llm_stats_cpdag_greedy</code>	0.173	33.6	0.454	2.63

The `pc_cpdag_llm` policy is consistently the cheapest active LLM policy per directed- F_1 point in every panel, often by an order of magnitude — the LLM is not asked to recover the skeleton, only to orient a small set of undirected edges, so the prompt is shorter and the action count smaller. Two action-count outliers are worth noting. Sonnet-4.6 `llm_stats_cpdag_greedy` averages 26.6 actions per episode against the budget of 40, and Haiku-4.5 `llm_stats_cpdag_greedy` averages 35.0 — both saturate the stats budget with statistical queries and still produce a weaker submission than the 3-action `pc_cpdag_llm` run on the same panel. Cost-per- F_1 is therefore not a soft proxy for capability: the policies that spend more on the stats path also score worse, on these models.

1144 F Trace exhibits

1145 This appendix gives concrete event-log views of agent behaviour behind the §5 headline numbers and
 1146 the per-cell aggregates of Appendix E. Each exhibit is one trace from a canonical `events.jsonl`
 1147 (Appendix G.2), selected by the key field that uniquely identifies a (panel, method, level, seed,
 1148 model) cell. We show five traces: one paired success/failure on the same L3 instance (F.F.2), and
 1149 three policy-contrast exhibits (F.F.3–F.F.5). The action vocabulary used in the tables is defined in
 1150 Appendix D.

1151 F.1 Trace schema and selection

1152 Each row in the trace tables corresponds to one `llm_action` event in `events.jsonl`, with
 1153 the immediately following `llm_tool_result` or `llm_intervention_result` summarised in
 1154 the *obs* column; the final row reports the *work_success* score for the submitted graph. The
 1155 *instance_metadata* event populates the table caption (true edges, CPDAG, intervention budget,
 1156 optimal-intervention set) but is not rendered as a row, and `llm_observation` / `llm_model_call`
 1157 events (which carry the prompt and the model’s full response) are folded into the *action* and *args*
 1158 columns. All independence_test calls use the global $\alpha = 0.05$ from §4.

1159 A trace is selected by a single grep on the panel’s events file:

1160 `key = "panel=active|method=<M>|level=<L>|seed=<S>|model=<X>"`

1161 Each row of every table below traces back to one event line under that key in the file path given by
 1162 Appendix G.2; the per-trace CSVs at `traces/aggregated/trace_*.csv` bridge the raw events to
 1163 these tables.

1164 F.2 Paired exhibit: GPT-5.5 vs Claude Haiku 4.5 on the same L3 instance

1165 Both runs see the same L3 instance ($d = 10$, $k = 12$): true DAG edges
 1166 $\{(0, 8), (0, 9), (1, 3), (1, 8), (4, 3), (4, 7), (5, 4), (5, 7), (6, 2), (6, 3), (6, 7), (9, 4)\}$, observational
 1167 CPDAG with 10 directed and 2 undirected edges $\{(0, 9), \{2, 6\}\}$, intervention budget 4, optimal-
 1168 intervention set $\{6, 9\}$. Both runs use `llm_raw` (no PC scaffold, no statistical tools).

step	action	args	obs
1	intervene	var=2, val=4.0	25 samples; budget left 3
2	intervene	var=6, val=2.5	25 samples; budget left 2
3	intervene	var=3, val=6.0	25 samples; budget left 1
4	intervene	var=4, val=4.0	25 samples; budget left 0
5	submit_graph	E_d =10, E_u =0	dir_f1=0.818, SHD=4

Table 3: GPT-5.5, `llm_raw`, L3, seed 362855691. Spends all 4 interventions on a coherent acyclic chain $\{2, 6, 3, 4\}$ — 6 matches the optimal set, 3 and 4 are downstream colliders) and recovers a 10-edge DAG at `directed_f1` = 0.818.

step	action	args	obs
1	intervene	var=3, val=10	25 samples; budget left 3
2	intervene	var=4, val=5.0	25 samples; budget left 2
3	submit_graph	E_d =9, E_u =0	dir_f1=0.000, SHD=19

Table 4: Claude Haiku 4.5, `llm_raw`, L3, seed 362855691 (same instance as Table 3). Submits after only 2 of 4 available interventions, on variables outside the optimal set; the resulting 9-edge submission scores `directed_f1` = 0.000 (all submitted directed edges are wrong-orientation or wrong-pair) with SHD = 19.

1169 GPT-5.5 exhausts its budget on a deliberately ordered intervention sequence and submits a graph
 1170 with 10 of 12 true edges correctly oriented. Haiku-4.5, on the same instance, terminates after 2
 1171 interventions on variables not in the optimal set $\{6, 9\}$ and submits a 9-edge graph in which no
 1172 submitted directed edge matches the true DAG. The contrast is not about the action vocabulary —

both agents have the same five tools and the same 4-intervention budget — but about how that budget is spent and whether the submission step is invoked before the evidence supports it.

F.3 CPDAG-guided LLM exhibit

step	action	args	obs
1	intervene	var=0, val=3.5	25 samples; budget left 2
2	intervene	var=2, val=2.5	25 samples; budget left 1
3	submit_graph	E_d =9, E_u =0	dir_f1=0.857, SHD=3

Table 5: GPT-5.5, pc_cpdag_llm, L3, seed 589330817. Instance: $d = 10$, $k = 12$, CPDAG with 9 directed and 3 undirected edges ($\{0, 2\}$, $\{0, 4\}$, $\{2, 6\}$), intervention budget 3, optimal set $\{2\}$.

Handed the PC-inferred CPDAG, the agent’s task collapses to orienting the 3 remaining undirected edges. Both interventions land on endpoints of those edges (0 and 2 — 2 is the optimal-set element), and the agent submits one step early with 1 intervention left over: $\text{directed_f1} = 0.857$, $\text{SHD} = 3$. The action mix shifts entirely away from correlation/partial_correlation (which the policy could call) toward direct interventions on under-determined nodes, then submission.

F.4 Statistical-tools exhibit

step	action	args	obs
1	intervene	var=1, val=4.0	25 samples; budget left 2
2	independence_test	i=0, j=2, cond={1}	indep=True, p=0.111
3	independence_test	i=0, j=1, cond={2,3,4,5,6,7,8,9}	indep=True, p=0.995
4	intervene	var=2, val=5.0	25 samples; budget left 1
5	independence_test	i=0, j=2, cond={1,3,4,5,6,7,8,9}	indep=True, p=0.964
6	submit_graph	E_d =9, E_u =0	dir_f1=0.762, SHD=5

Table 6: GPT-5.5, llm_stats, L3, seed 1877150515. Instance: $d = 10$, $k = 12$, intervention budget 3, optimal set $\{0\}$.

The agent interleaves interventions with three Fisher- z independence tests: an unconditional-style test on $(0, 2 \mid \{1\})$, a high-conditioning-set test on $(0, 1 \mid \text{rest})$ that rules out a direct edge, and a follow-up on $(0, 2)$ after intervening on 2. All three return $\text{indep}=\text{True}$; the final submission omits direct edges between 0 and either 1 or 2, consistent with what the tests indicated. The trace shows what the agent does *with* statistical tools — not that they win on this seed (the llm_raw run on this instance scores within 0.012).

F.5 Scale exhibit at L4

step	action	args	obs
1	intervene	var=1, val=3.0	25 samples; budget left 5
2	intervene	var=6, val=10.0	25 samples; budget left 4
3	intervene	var=0, val=3.0	25 samples; budget left 3
4	intervene	var=2, val=2.5	25 samples; budget left 2
5	intervene	var=3, val=3.0	25 samples; budget left 1
6	intervene	var=5, val=2.0	25 samples; budget left 0
7	submit_graph	E_d =8, E_u =0	dir_f1=0.636, SHD=7

Table 7: GPT-5.5, llm_raw, L4, seed 511033471. Instance: $d = 12$, $k = 14$, intervention budget 6, optimal-intervention set $\{0, 1, 4, 5\}$.

At L4 the agent uses all 6 interventions before submitting. Three of the six chosen variables ($\{0, 1, 5\}$) match the optimal set; $\{2, 3, 6\}$ do not. The submission has 8 of 14 true edges correctly oriented ($\text{directed_f1} = 0.636$, $\text{SHD} = 7$), a measurable drop from the L3 llm_raw success in Table 3 (0.818) on a comparable budget per node.

1193 F.6 Reproducing or extending these traces

1194 The five trace CSVs are at `traces/aggregated/trace_1_*.csv` through `trace_5_*.csv`
 1195 in the repository (Appendix G.5). They are derived from the `panel_events.jsonl` files via
 1196 `scripts/extract_trace_rows.py`, invoked as e.g.

```
1197 python scripts/extract_trace_rows.py \
1198     --events traces/ladder/full_gpt55/events.jsonl \
1199     --key "panel=active|method=llm_raw|level=3|seed=362855691|model=gpt-5.5" \
1200     --out traces/aggregated/trace_1.csv
```

1201 Each `llm_model_call` event in the source file additionally carries the model's
 1202 `parsed_action.reasoning_summary` and the full prompt/response, which are not ren-
 1203 dered above but are available for any reviewer who wants to read the agent's prose for a given
 1204 step.

1205 G Reproducibility

1206 This appendix gives the exact command, model strings, seed manifest, and software environment
 1207 needed to rerun the experiments of Section 4. Per-panel results are reported separately in Appendix E;
 1208 this section is restricted to what a reviewer needs to regenerate the run inputs.

1209 G.1 Reproduction command

1210 The canonical invocation, anchored on the GPT-5.5 panel, is:

```
1211 LADDER_MAX_RETRIES=20 uv run python run_ladder.py \
1212     --levels 0,1,2,3,4,5 \
1213     --seeds-per-level 8 \
1214     --models gpt-5.5 \
1215     --alpha 0.05 \
1216     --max-steps-raw 20 \
1217     --max-steps-stats 40 \
1218     --preflight-seed 20260422 \
1219     --out-dir traces/ladder/full_gpt55
```

1220 Substituting the `-models` string from the table in Appendix G.2 and changing `-out-dir` to the
 1221 matching `Trace` directory listed there reproduces the corresponding panel. The `env-var` prefix
 1222 `LADDER_MAX_RETRIES=20` enables an auto-resume loop (`run_ladder.py`: 1289–1304) that retries
 1223 on transient provider errors, restoring from the on-disk checkpoint after a 10-second backoff. The
 1224 default value is 0 (no retry), so the `env-var` is required to match the canonical runs.

1225 G.2 Model versions

1226 The model string and run identifier for each panel are lifted verbatim from `run_manifest.json`
 1227 in the corresponding output directory. All five runs share `levels=[0,1,2,3,4,5]`,
 1228 `seeds_per_level=8`, $\alpha = 0.05$, `max_steps_raw = 20`, and `max_steps_stats = 40`.

Panel	Model string	Provider	run_id (UTC)	Trace directory
GPT-5.5	<code>gpt-5.5</code>	OpenAI	20260503T183924Z	<code>traces/ladder/full_gpt55</code>
GPT-5.4-mini	<code>gpt-5.4-mini</code>	OpenAI	20260504T054452Z	<code>traces/ladder/full_gpt54mini</code>
Claude Sonnet 4.6	<code>openrouter/anthropic/claude-sonnet-4-6</code>	OpenRouter	20260504T132941Z	<code>traces/ladder/full_sonnet46_or</code>
Claude Haiku 4.5	<code>openrouter/anthropic/claude-haiku-4.5</code>	OpenRouter	20260504T111120Z	<code>traces/ladder/full_haiku45_or</code>
Gemini 3 Flash	<code>google/gemini-3-flash-preview</code>	LiteLLM	20260504T172409Z	<code>traces/ladder/full_gemini3flash</code>

1231 Model strings without a provider prefix are dispatched by `provider_for_model` to the OpenAI
 1232 or Anthropic native APIs; prefixed strings (`anthropic/...`, `openrouter/...`, `google/...`) are
 1233 routed through LiteLLM. The Sonnet and Haiku panels use the `_or` suffix to mark the canonical
 1234 OpenRouter re-runs that supersede earlier partial runs at the same `run_id`; the aggregated metrics

1235 in `traces/aggregated/per_model_per_level_per_method.csv` are sourced exactly from the
1236 five directories listed above.

1237 G.3 Seed manifest

1238 The same accepted-seed map is reused across all five panels — it appears verbatim in every
1239 `run_manifest.json` — so cross-model comparisons are paired by instance.

- 1240 • **L0:** 1276453582, 27998312, 931551983, 974369856, 1539978433, 868078822,
1241 452641024, 963592850
- 1242 • **L1:** 1257633785, 1100146725, 1910407594, 746528573, 1175759265,
1243 1982942393, 1449442515, 1479720101
- 1244 • **L2:** 823847298, 834246677, 32015887, 716908331, 1426457537, 684342303,
1245 1207468583, 1612783012
- 1246 • **L3:** 1329552115, 160719912, 362855691, 589330817, 1877150515,
1247 370845052, 976218778, 1286035187
- 1248 • **L4:** 518645330, 511033471, 1198956094, 154587426, 1368642322,
1249 1163629188, 1332933138, 1231061499
- 1250 • **L5:** 1526642568, 1170708238, 385840006, 32447648, 1017236502,
1251 1717602974, 480184089, 2015902496

1252 The map is generated by `build_seed_map_preflight` (`run_ladder.py:189-217`) with
1253 `-preflight-seed 20260422`, accepting a candidate seed only if it passes the rejection filters
1254 of Appendix A.3, then frozen and replayed via the manifest. Pinning `-preflight-seed` together
1255 with the filter spec is sufficient to regenerate the map from scratch; copying the 48 integers above is
1256 provided only for direct replay.

1257 G.4 Software environment

- 1258 • Python ≥ 3.12 , declared in `pyproject.toml`.
- 1259 • Install via `uv sync`, which materialises the locked dependency set.
- 1260 • LLM dispatch goes through `litellm` ≥ 1.79 (with `openai` ≥ 2.30 for native OpenAI
1261 calls). PC inference uses `causal-learn` $\geq 0.1.4.5$. Orchestration relies on `numpy`, `pandas`
1262 $\geq 3.0.2$, `python-dotenv` $\geq 1.2.2$, and `tenacity` $\geq 9.1.4$.
- 1263 • Provider credentials `OPENAI_API_KEY`, `ANTHROPIC_API_KEY`, and
1264 `OPENROUTER_API_KEY` are read from `.env` at startup (`run_ladder.py:1069-1095`); the
1265 runner refuses to launch if a key required by the requested model panel is missing.

1266 G.5 Repository

1267 The full source, manifests, and per-panel `events.jsonl` traces are available at [https://](https://anonymous.4open.science/r/ACDB)
1268 `anonymous.4open.science/r/ACDB`.